

AI and Intelligent Transport System (ITS)

18 May 2026

B. Saovapakhiran, NECTEC

Part I : AI Vision in ITS Paradigm

Agenda

- Section 1
 - AI Vision and ITS Paradigm
- Section 2
 - State-of-the-Art Object Detection
- Section 3
 - Implementation & Deployment

Section 1 : AI Vision in ITS

ITS Legacy Bottleneck

→ Legacy ITS technologies

- ◆ Sensor-based devices
- ◆ Low-bandwidth consumption
- ◆ Multi-hop communications
- ◆ Images are not the main stream data for processing information

→ Infrared Sensor in Modern Traffic Management

- ◆ Requires a large number of sensors and associate devices.
- ◆ High installation cost and expensive deployment
- ◆ Limitation in visioning object and deep-dive into object features.

Shift to Vision-AI

- Camera devices become more cheaper and more accessible.
- Camera can be seen as a “Sensor” Device.
- “One” Camera can represent “Multi-modal” Sensors.
 - ◆ Object Detection
 - ◆ Object Recognition
 - ◆ Object Identification
- “One” Camera can interpret information and be used in “Multi” Applications.
 - ◆ Automated Plate Recognition
 - ◆ Congestion Monitoring
 - ◆ Intelligent Parking

Architectural Case Studies

→ “Passive Sensing” Concept in ITS

- ◆ M-PASS
- ◆ Easy-PASS
- ◆ Intelligent Parking System

→ “Privacy” Concept in ITS

- ◆ Edge Nodes => NVIDIA Jetson Series
- ◆ Edge Computing
- ◆ Federated Approach

→ “Centralized Decision Data”

- ◆ Data Aggregation on Centralized Server (on Cloud)
- ◆ Dashboard / Metadata / Traffic Heatmap from 100x Edge nodes
- ◆ Automated Government Services (Ticket / Fee / Accident Reports)

Architectural Case Studies



“Passive Sensing” Concept in ITS

- M-PASS
- Easy-PASS
- Intelligent Parking System



“Privacy” Concept in ITS

- Edge Nodes (NVIDIA Jetson)
- Edge Computing
- Federated Approach



“Centralized Decision Data”

- Data Aggregation on Cloud Server
- Dashboard & Traffic Heatmaps
- Automated Govt Services (Tickets/Reports)

Bandwidth for CCTVs

- Modern ITS requires high resolution image data for tasks : classification or Re-Identification
 - ◆ Resolution 4K (on Highway / Parking Lots)
 - ◆ Standard Frame Rate 30 FPS (required to capture high speed moving object without image blurring)
 - ◆ Standard Codec H.265

→ Bit Rate (4K + 30FPS) = 8 Mbps

Aggregated Uplink Bandwidth

→ For 100 Cameras

$$\begin{aligned} \rightarrow \quad \text{Total Bandwidth (Mbps)} &= (\text{Bit Rate per Cams}) \times (\text{Number of Cams}) \\ &= 800 \text{ Mbps} \end{aligned}$$

→ Reality : Standard Fiber Optic (Normally 1 Gbps UL)

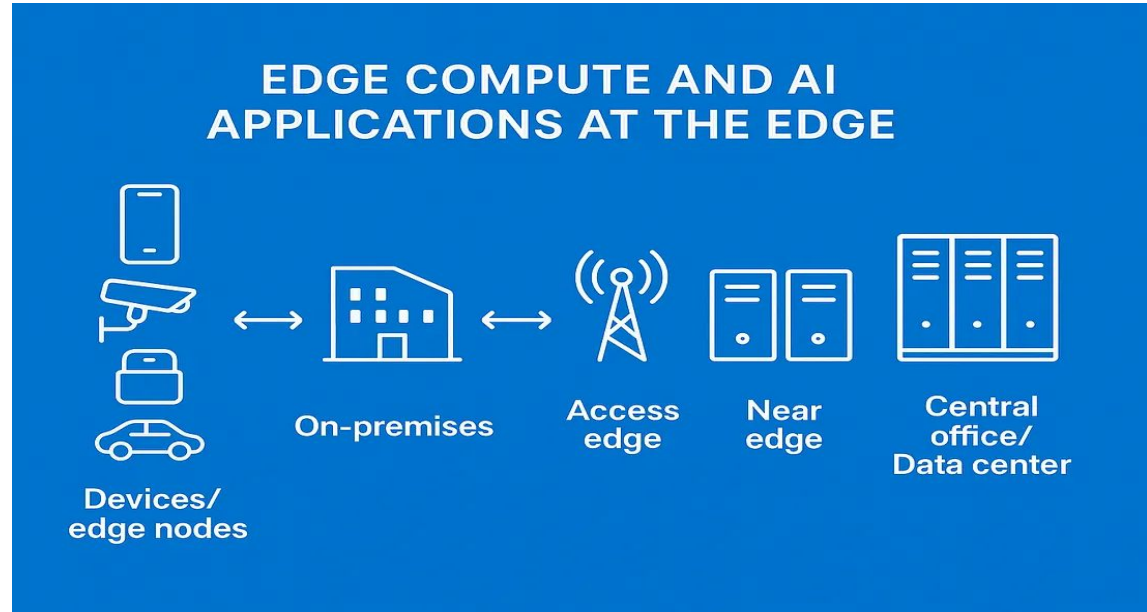
→ For 1,000 Cameras (City Scale Deployment)

→ Bandwidth Requirement = 8 Gbps

- ◆ Infeasible in deployment / implementation
- ◆ Too expensive costs : Monthly Transfer can up to 2,500,000 GB/month with Cost = 22,000 THB per month**

Edge Computing

- Distributing Edge Compute Nodes to Data Sources ACAP (As Closed As Possible)



Edge Computing

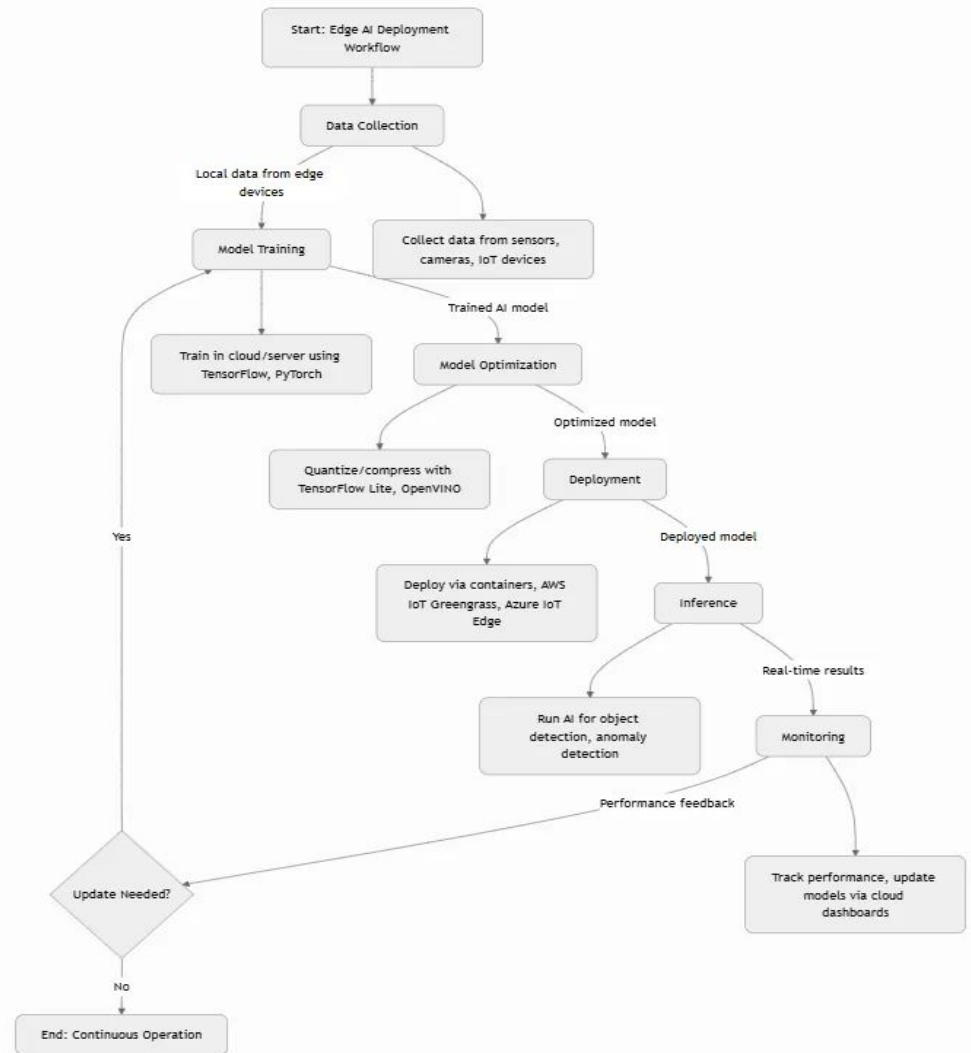
- Distributing Edge Compute Nodes to Data Sources ACAP (As Closed As Possible)
- Detecting the Vehicle at Edge / Capturing the necessary data / Sending it to Cloud
- What should we have at Edge nodes ?
 - ◆ Hardware for AI Inference
 - ◆ AI model for “Vehicle Object Detection”
 - ◆ Inference Data would be like “Vehicle, White Toyota Vios, arrives at “xxxx” timestamp and exits at “xxxx” timestamp.
- Gain : From Video 8 Mbps downto < 2KB JSON + < 4KB Image

Trade-off: Edge vs. Cloud Computing

Feature	Edge Computing	Cloud Computing
Latency	Ultra-low; immediate response at data source.(< 20 ms)	Higher; depends on network distance and traffic. (100 - 300 ms usual)
Bandwidth	Local processing reduces data sent to network.(< 2 KB JSON)	Requires high bandwidth to transmit raw data. (8 Mbps per camera for 4K image)
Processing Power	Limited; specialized hardware for specific tasks. (Optimized AI model is preferable)	Scalable; virtually unlimited computational power. Larger AI Model is feasible. (Smarter Model)
Network Resilience	More Robustness and Easier in Data Recoveries	Large amount of data can be lost and hard for recovery due to larger size.

Edge Deployment WorkFlow

- Data Collection
- Model Training
- Model Optimization
- Model Deployment
- Model Inference
- Performance Monitoring
- Update Phase*



Edge AI Hardware Examples

- Hailo-8 AI Accelerator
- Kinara AI Accelerator
- RPI + AI Accelerator
- RK3588 + Hailo-8
- Kinara Ara-2 + MX8M+
- NVIDIA Jetson

Top Hardware options for Edge AI Deployment

Hailo-8 AI Accelerator



Kinara AI Accelerator



RPI + AI Accelerators



NVIDIA Jetson

AI on Boards, Edge Box



RK3588 + Hailo -8

AI on Modules & Boards, Edge Box



i.MX 8M Plus + Kinara Ara-2

AI on Modules & Boards, Edge Box

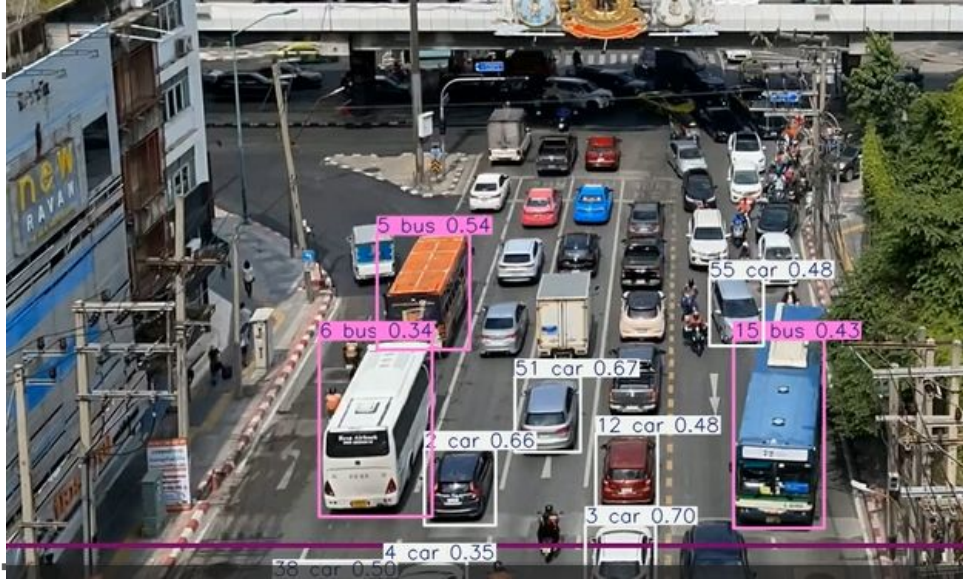


NVIDIA® Jetson Module Powered Devices Comparison

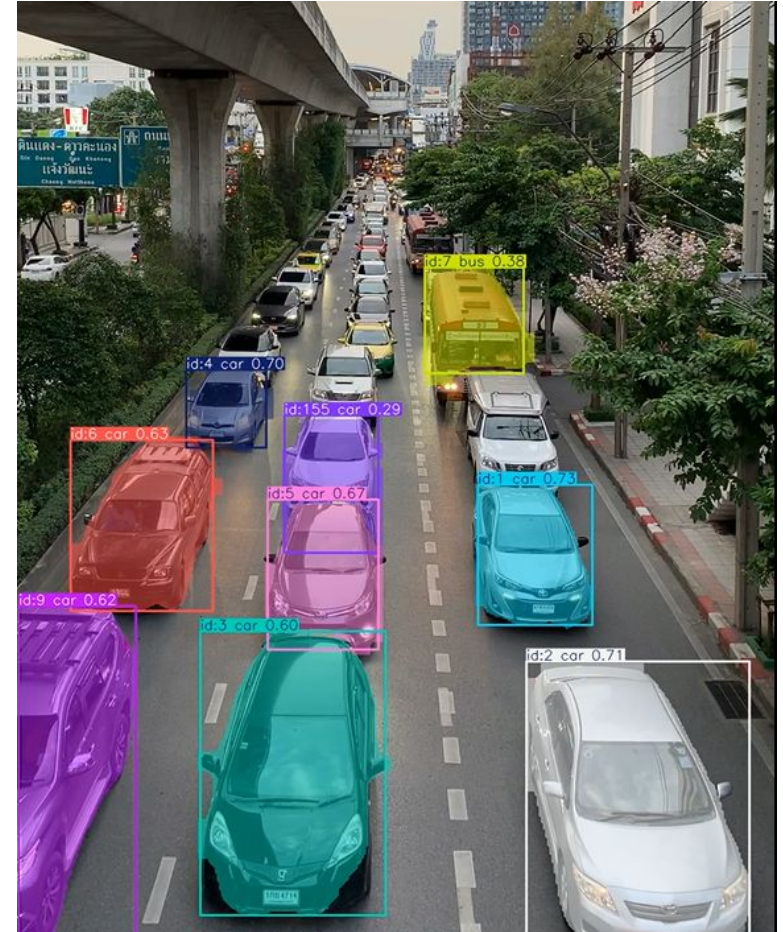
	Jetson Nano			Jetson Xavier NX				Jetson Orin NX		Jetson AGX Xavier		Jetson AGX Orin		
Device								 Coming soon						
	reComputer J1010	reComputer J1020	NVIDIA® Jetson Nano Developer Kit-B01	reComputer J2011	reComputer J2012	Jetson SUB Mini-PC Black	NVIDIA® Jetson Xavier NX Developer Kit	reComputer J4011	reComputer J4012	Jetson AGX Xavier H01 Kit	NVIDIA® Jetson AGX Xavier Dev Kit	NVIDIA® AGX Orin™ Dev Kit	reServer J5014	
AI Performance	472 GFLOPS			21 TOPS				70 TOPS	100 TOPS	32 TOPS		275 TOPS	275 TOPS	
GPU	NVIDIA Maxwell™ architecture with 128 NVIDIA CUDA® cores			NVIDIA Volta™ architecture with 384 NVIDIA CUDA® cores and 48 Tensor cores				NVIDIA Ampere architecture with 1024 NVIDIA® CUDA® cores and 32 tensor cores		NVIDIA Volta™ architecture with 512 NVIDIA® CUDA® cores and 64 Tensor cores		NVIDIA Ampere architecture with 2048 NVIDIA® CUDA® cores and 64 Tensor Cores		
CPU	Quad-Core Arm® Cortex®-A57 MPCore processor			6-core NVIDIA Carmel ARM®v8.2 64-bit CPU 6 MB L2 + 4 MB L3				6-core Arm® Cortex®-A78AE v8.2 64-bit CPU 1.5MB L2 + 4MB L3	8-core Arm® Cortex®-A78AE v8.2 64-bit CPU 2MB L2 + 4MB L3	8-core NVIDIA Carmel Arm®v8.2 64-bit CPU 8MB L2 + 4MB L3		12-core Arm® Cortex®-A78AE v8.2 64-bit CPU 3MB L2 + 6MB L3		
Memory	4 GB 64-bit LPDDR4 25.6GB/s			8 GB 128-bit LPDDR4x 59.7GB/s	16 GB 128-bit LPDDR4x 59.7GB/s	8 GB 128-bit LPDDR4x 59.7GB/s	8 GB 128-bit LPDDR4x 51.2GB/s	8GB 128-bit LPDDR5 102.4 GB/s	16GB 128-bit LPDDR5 102.4 GB/s	32 GB 256-bit LPDDR4x 136.5GB/s		32GB 256-bit LPDDR5 204.8 GB/s	64GB 256-bit LPDDR5 204.8 GB	
Storage	16 GB eMMC 5.1		microSD (card not included)	16 GB eMMC 5.1		16 GB eMMC 5.1 5*SATA (256GB SATA included)	microSD (card not included)	Supports external NVMe	Supports external NVMe	32 GB eMMC 5.1		64GB eMMC 5.1		
Video Encode	1*4K30 2*1080p60 4*1080p30 4*720p60 9*720p30 (H.265 & H.264)		1*4K30 4*1080p30 9*720p30 (H.264/H.265)	2*4K60 4*4K30 10*1080p60 22*1080p30 (H.265) 2*4K60 4*4K30 10*1080p60 20*108p30 (H.264)			2*4K30 6*1080p60 14x1080p30 (H.265/H.264)	1*4K60 3*4K30 6*1080p60 12*1080p30 (H.265) H.264, H.265, AV1		4*4K60 8*4K30 16*1080p60 32*1080p30 (H.265) 4*4K60 8*4K30 14*1080p60 30*1080p30 (H.264)	2*4Kp60 HEVC/2*4Kp60 12-Bit Support	2x 4K60 4x 4K30 8x 1080p60 16x 1080p30 (H.265) H.264, AV1		
Video Decode	1*4K60 2*4K30 4*1080p60 8*1080p30 9*720p60 (H.265 & H.264)		1*4K60 2*4Kp30 8*1080p30 18*720p30 (H.264/H.265)	2*8K30 6*4K60 12*4K30 22*1080p60 44*1080p30 (H.265) 2*4K60 6*4K30 10*1080p60 22*1080p30 (H.264)			2*4K60 4*4K30 12*1080p60 32*1080p30 (H.265) 2*4K30 6x1080p60 16x1080p30 (H.264)	1*8K30 2*4K60 4*4K30 9*1080p60 18*1080p30 (H.265) H.264, H.265, VP9, AV1		2*8K30 6*4K60 12*4K30 26*1080p60 52*1080p30 (H.265) 4*4K60 8*4K30 16*1080p60 32*1080p30 (H.264)		1x 8K30 3x 4K60 7x 4K30 11x 1080p60 22x 1080p30 (H.265) H.264, VP9, AV1		
Networking	1*RJ45 Gigabit Ethernet Connector (10/100/1000)					2*RJ45 Gigabit Ethernet Connector (10/100/1000)		1*RJ45 Gigabit Ethernet Connector (10/100/1000)					1*RJ45 10 Gigabit Ethernet Connector	1*RJ45 10 Gigabit Ethernet Connector 1*RJ45 Gigabit Ethernet Connector

Section 2 : SOTA Object Detection

Object Detection Task in ITS



- Two sub-problem :
- Where is the vehicle ? (Boundary Box)
- What is the object ? (Classification)



Object Detector Types

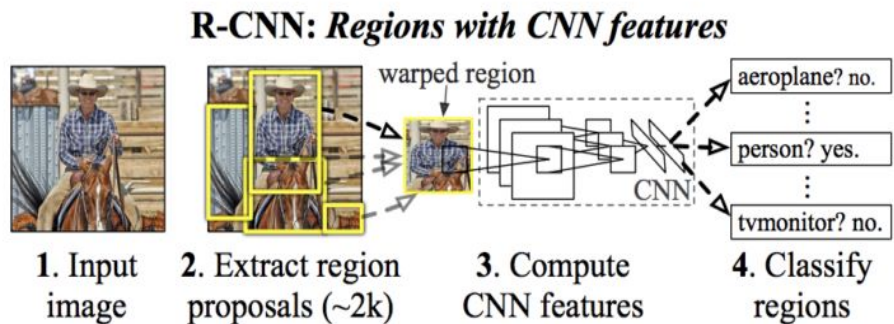
- Two-Stage Object Detector Model : Fast R-CNN, ...
- One-Stage Object Detector Model : YOLO, ...

Two-Stage Detector

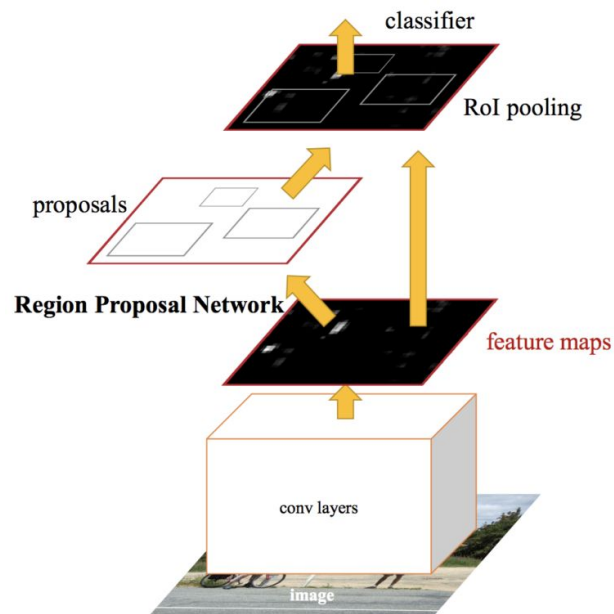
- Example : Fast R-CNN
- Region Proposal Network (RPN)
 - ◆ The model determines the area that is likely to be Blob object.
 - ◆ “Where are the objects?”
- Feature Extraction and Classification
 - ◆ The model predicts the object class or
 - ◆ “Which class should this object be in?”

Fast R-CNN

→ Original R-CNN



→ Faster R-CNN



One-Stage Detector

- More Simpler Approach : Unifying everything at once !
- Divide the image into $S \times S$ grid :
- The model perform the followings at the same time : :
 - ◆ Predict the coordinates of the Bounding Box
 - ◆ Predict the confidence score of the area
 - ◆ Predict the class probability such as “Car with. 80% probability”
- Example : YOLO

YOLO (You-Only-Look-Once)

- One-Stage Detector (Unified Detection)
- A Single Neural Network to predict bounding boxes and class probabilities at the same time.
- Real-time object detection > 45 FPS (First version)
- Fast YOLO can achieve > 155 FPS (while mAP is better than other detectors)
- Faster than most two-stage detection, e.g. Fast R-CNN

YOLO Architecture

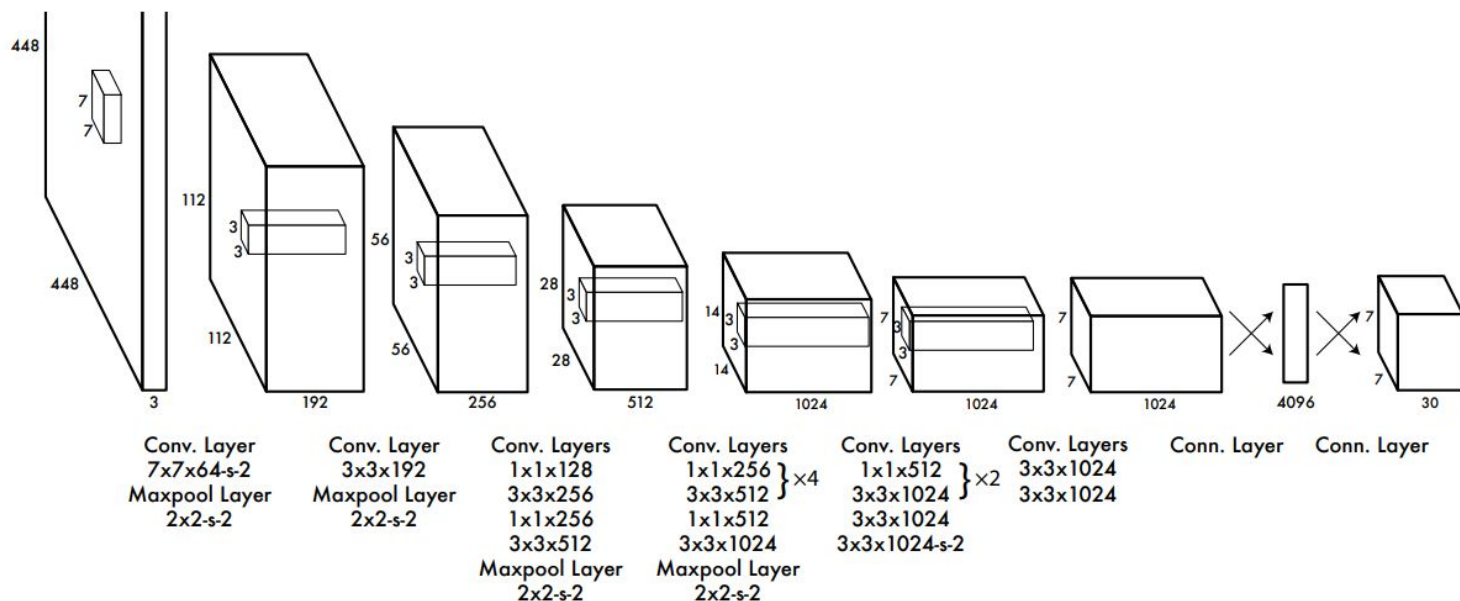


Figure 3: The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224×224 input image) and then double the resolution for detection.

→ You Only Look Once: Unified, Real-Time Object Detection , Joseph Redmon*, Santosh Divvala*†, Ross Girshick¶, Ali Farhadi*†

Why YOLO is so popular?

- Speed
- Detection Accuracy
- Generalization (Fast & Robust)
- Open Source

YOLO Speed / Accuracy Comparison

Table 6. Performance of YOLO, Faster R-CNN and SSD methods works metrics speed and accuracy using the test set PASCAL VOC 2007 [24].

Detection Frameworks	Train	mAP	FPS
Fast R-CNN	2007+2012	70.0	0.5
Faster R-CNN VGG-16	2007+2012	73.2	7
Faster R-CNN ResNet	2007+2012	76.4	5
YOLO	2007+2012	63.4	45
SSD300	2007+2012	74.3	46
SSD500	2007+2012	76.8	19
YOLO v2 288 x 288	2007+2012	69.0	91
YOLO v2 352 x 352	2007+2012	73.7	81
YOLO v2 416 x 416	2007+2012	76.8	67
YOLO v2 480 x 480	2007+2012	77.8	59
YOLO v2 544 x 544	2007+2012	78.6	40

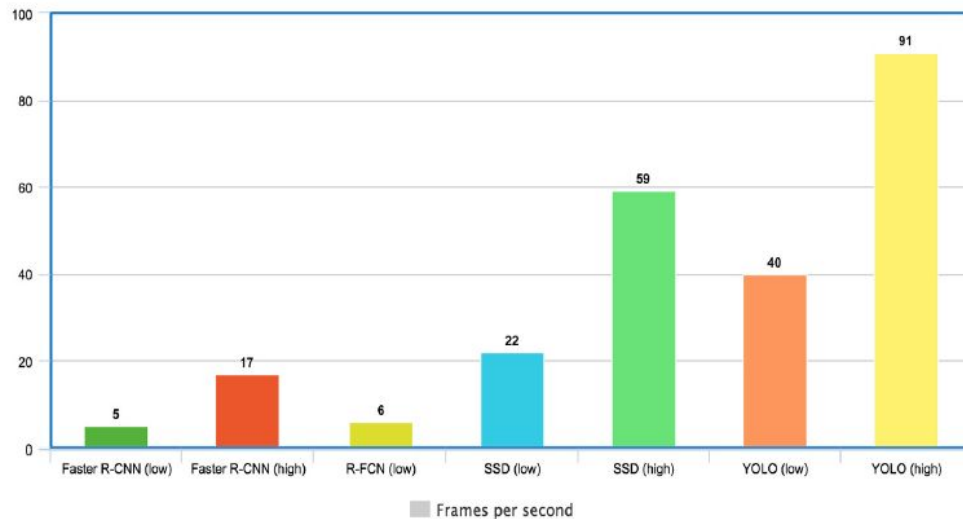


Figure 6. Comparison of frames processed per second (FPS) implementing the Faster R-CNN, R-FCN, SSD and YOLO models using input images with different resolutions.

How does YOLO work?

→ Each Grid “predict” =>

- ◆ B BoundingBox + Confidence Score

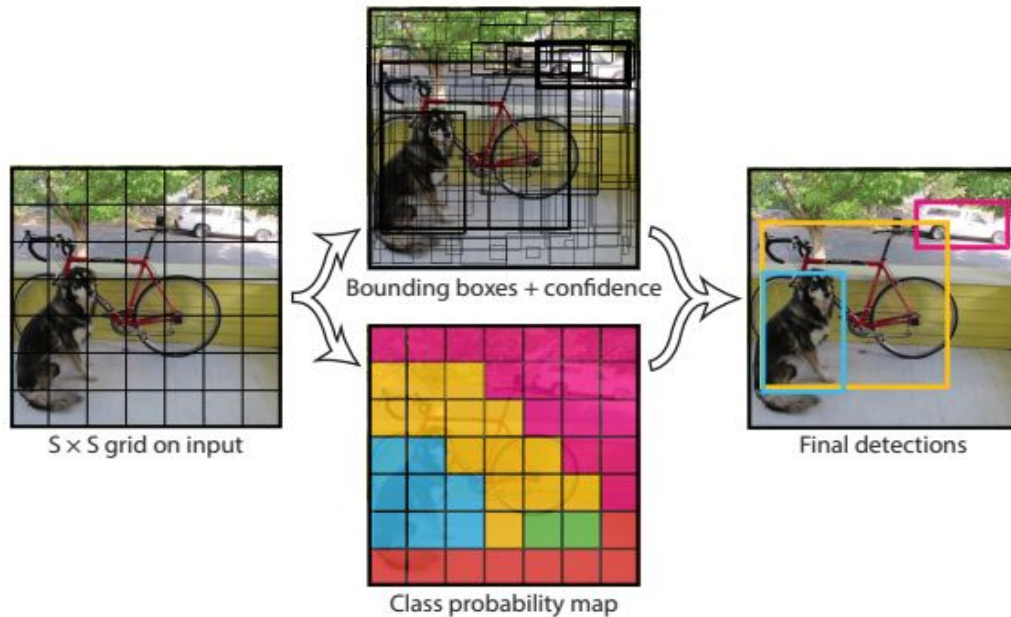
- ◆ $\Pr(\text{Object}) * \text{IoU}$

→ Each Grid “predict” =>

- ◆ $\Pr(\text{Class}_i | \text{Object})$

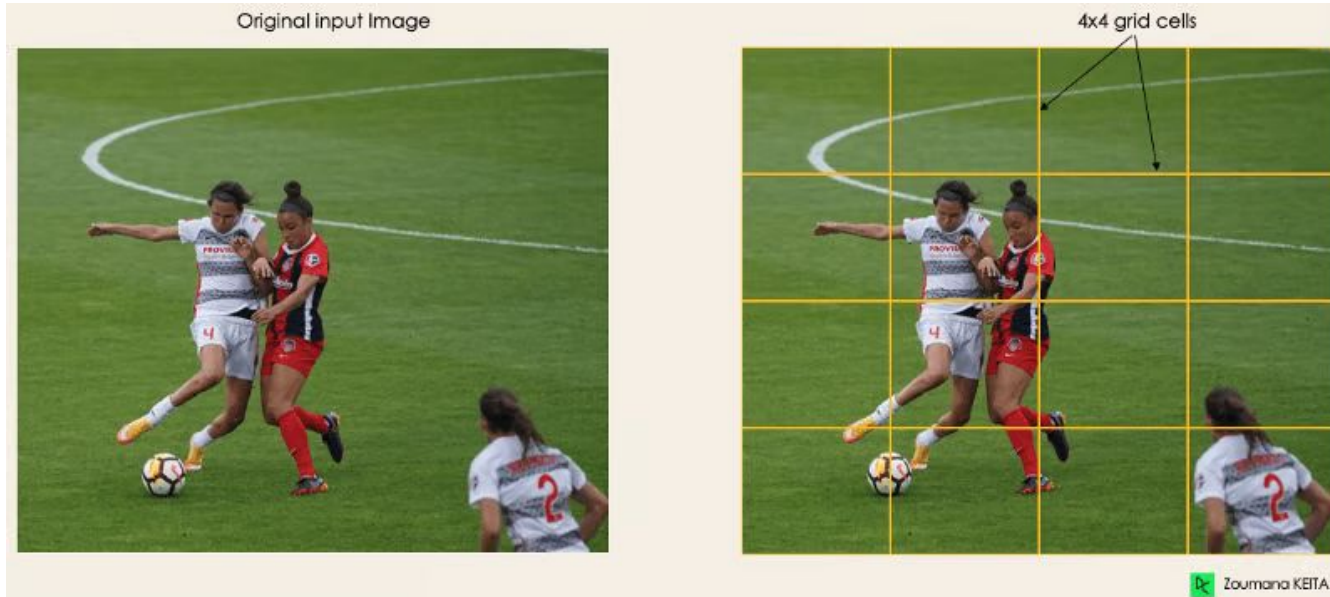
→ Final Detection =>

- ◆ $\Pr(\text{Class}_i | \text{Object}) * \Pr(\text{Object}) * \text{IoU}$



How does YOLO work?

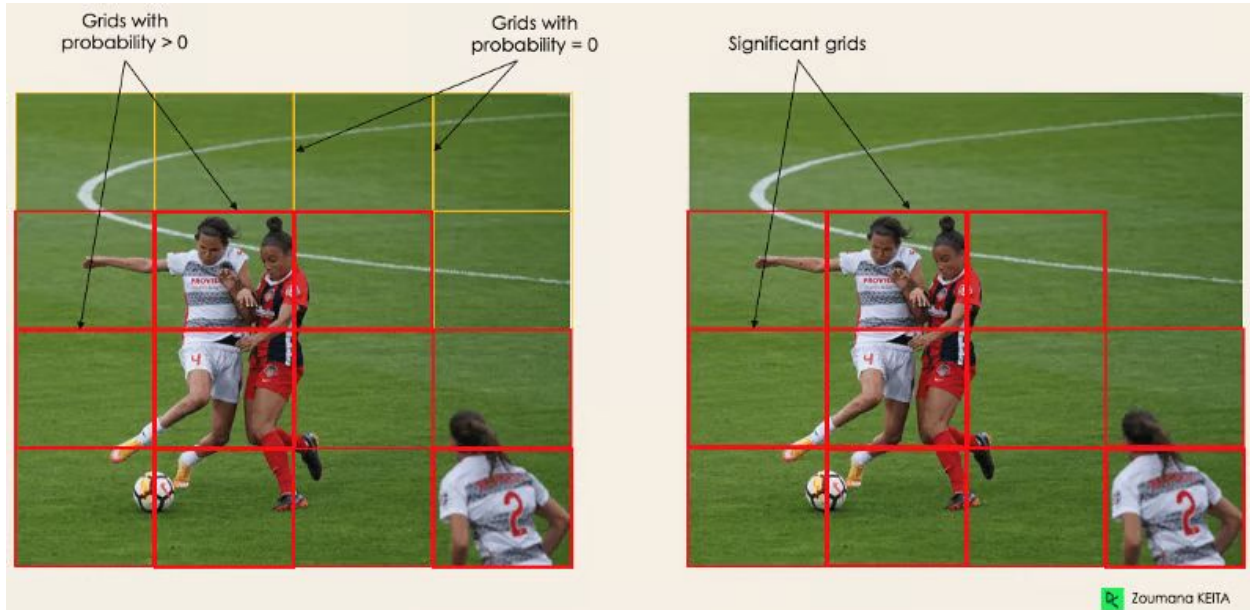
→ Divide the input image into NxN Grid Cells



→ <https://www.datacamp.com/blog/yolo-object-detection-explained>

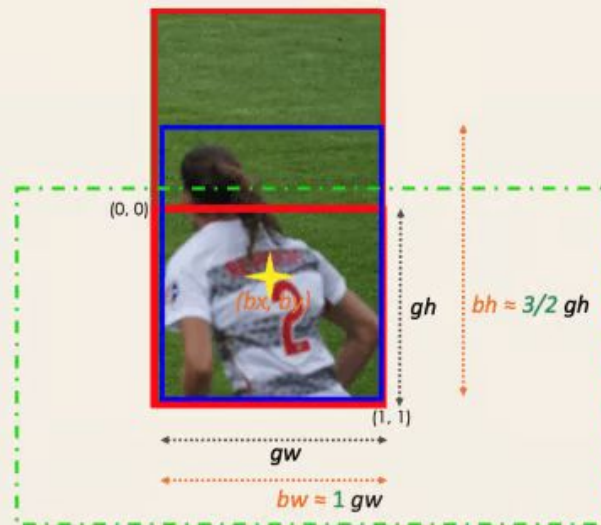
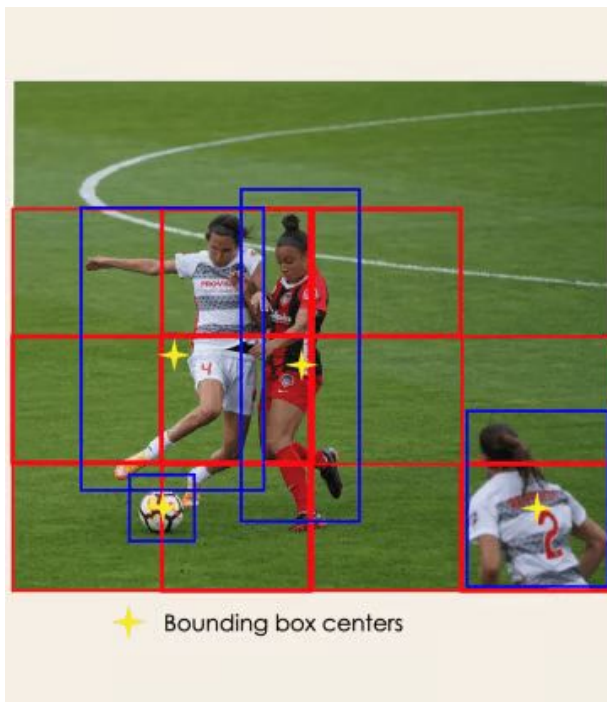
How does YOLO work?

→ BBox Regression : $Y = [pc, bx, by, bh, bw, c1, c2]$



→ <https://www.datacamp.com/blog/yolo-object-detection-explained>

How does YOLO work?



From the previous info we can have for e.g.
 $Y = [1, bx, by, 3/2, 1, c1, c2]$

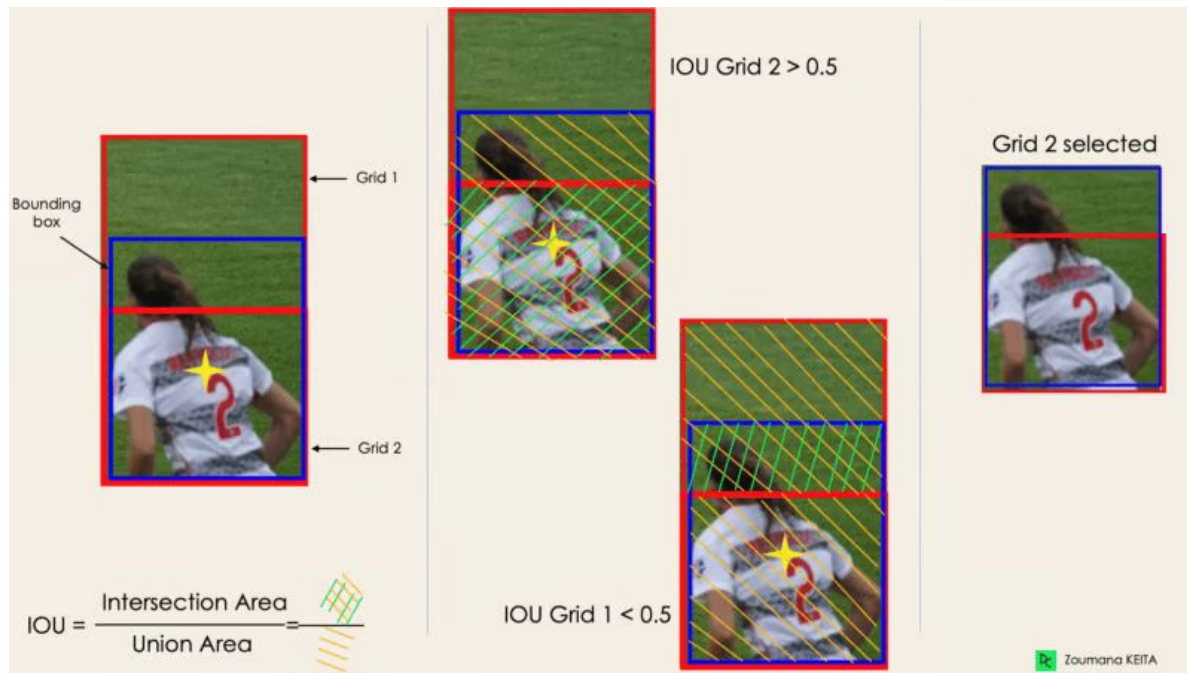
- First 1 means 100% of object presence

- gh, gw : height & width of the grid
- $0 \leq bx \leq 1$
- $0 \leq by \leq 1$
- bh and bw can be more than 1

How does YOLO work?

→ IoU Threshold

- ◆ Use to define how high relevant Grid Cell should be to detect the object.



→ <https://www.datacamp.com/blog/yolo-object-detection-explained>

Non-Max Suppression or NMS

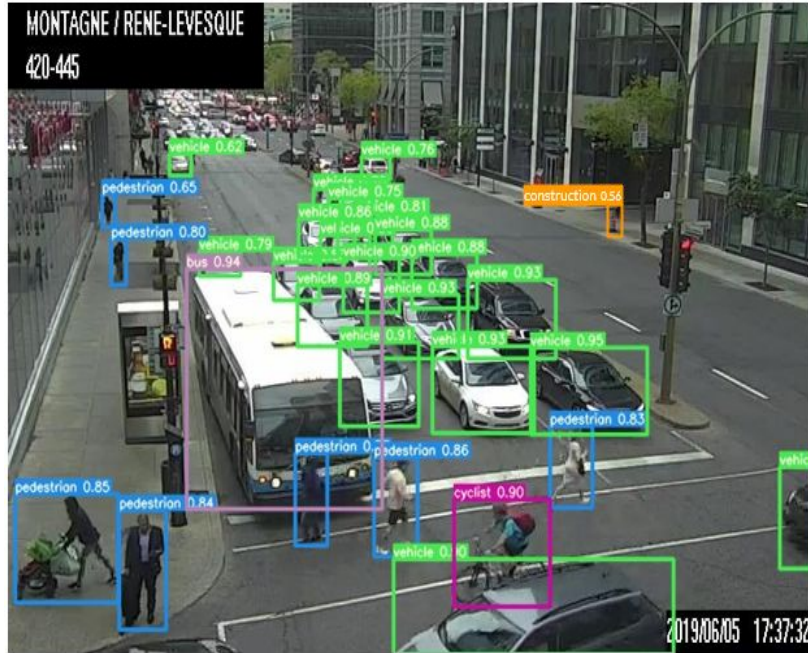
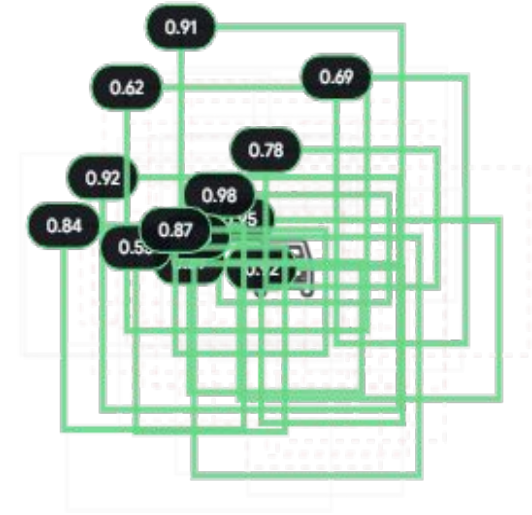


Figure From : An AI-Based Traffic Light Control System Using YOLOv10 with Emergency and Public Bus Prioritization: A Case Study in Baghdad



Green = Kept | Red = Overlap Suppressed | Gray = Low Confidence

→ CCTV ⇒ GPU (YOLO Inference 5ms) <mismatched> CPU (NMS > 10 ms) ⇒ Final Detection

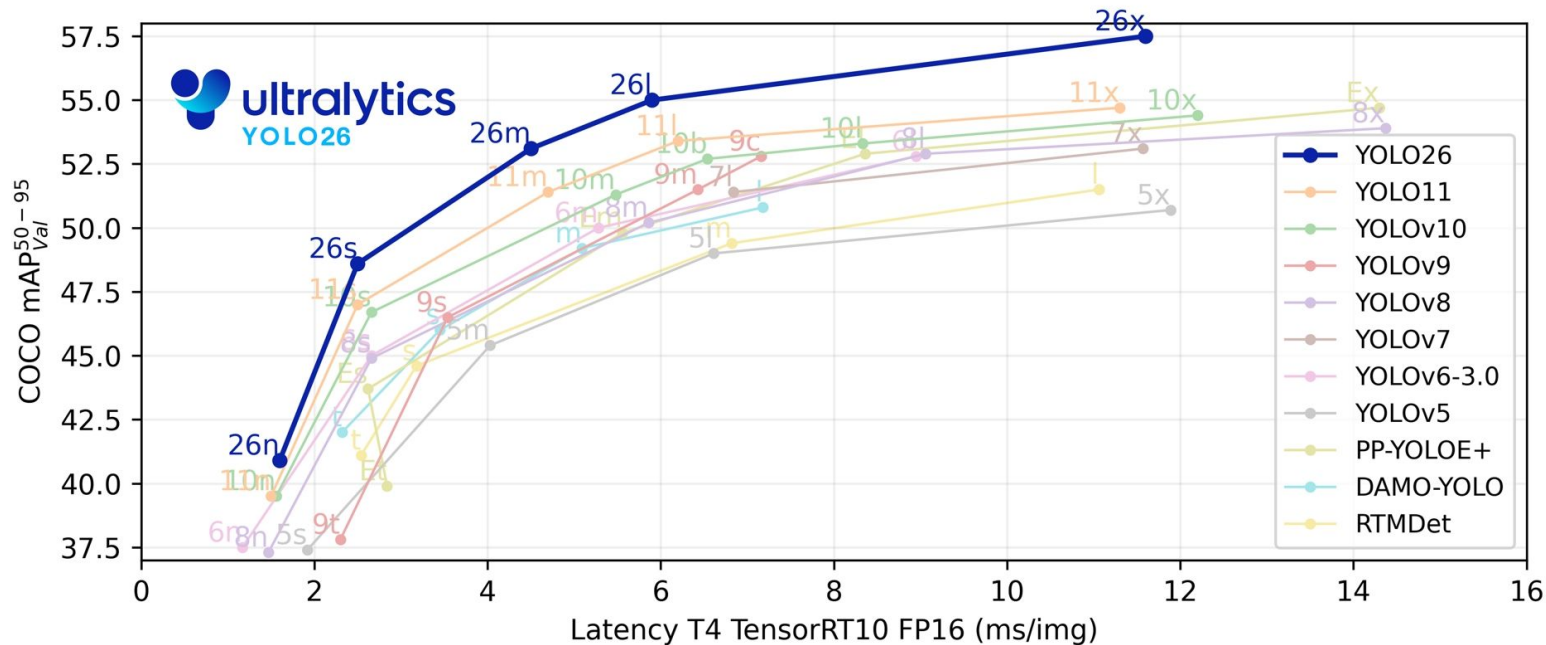
NMS Bottleneck

- NMS Latency increase as the number of objects increase
- Empty Street
 - ◆ 0 cars
 - ◆ $\text{NMS} < 1\text{ms}$
- 6PM at Asoke Intersection
 - ◆ 150 overlapping motorcycles + cars
 - ◆ >1000 candidate boxes
 - ◆ $\text{NMS} > 30\text{ms}$
 - ◆ Performance dropped significantly!
- This happens for YOLO version < 10
 - ◆ The bottleneck solved in YOLOv10, v11, v12 (NMS-Free architecture)

YOLO Timeline



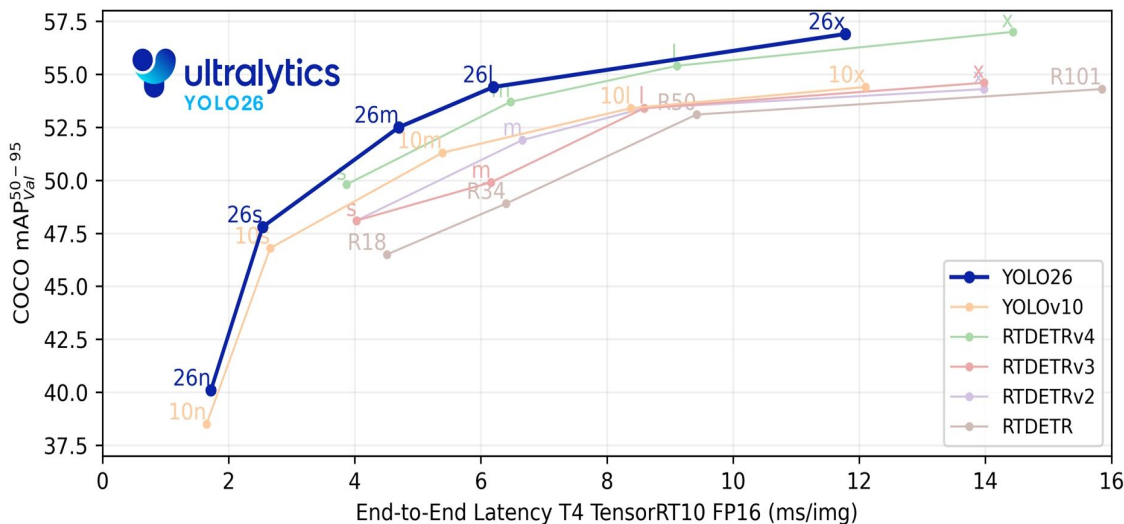
YOLO performance



<https://docs.ultralytics.com/models/yolo26>

YOLO 2026 (<https://docs.ultralytics.com/models/yolo26>)

- Native End-to-End Model
- NMS-Free Model (First solved in YOLO10 by Ao Wang at Tsinghua University)
- MuSGD Optimizer (Hybrid between SGD and Muon)
- 43% Faster on CPU
- ROC better than RT-DETR (Real-Time Detection Transformer)



RT-DETR

→ Real-Time
Variant of
DETR
(DeTectiion
TRansformer)
based model.

.08069v3 [cs.CV] 3 Apr 2024

DETRs Beat YOLOs on Real-time Object Detection

Yian Zhao^{1,2†} Wenyu Lv^{1†‡} Shangliang Xu¹ Jinman Wei¹ Guanzhong Wang¹
Qingqing Dang¹ Yi Liu¹ Jie Chen^{2✉}

¹Baidu Inc, Beijing, China ²School of Electronic and Computer Engineering, Peking University, Shenzhen, China

zhaoyian@stu.pku.edu.cn lvwenyu01@baidu.com jiechen2019@pku.edu.cn

Abstract

The YOLO series has become the most popular framework for real-time object detection due to its reasonable trade-off between speed and accuracy. However, we observe that the speed and accuracy of YOLOs are negatively affected by the NMS. Recently, end-to-end Transformer-based detectors (DETRs) have provided an alternative to eliminating NMS. Nevertheless, the high computational cost limits their practicality and hinders them from fully exploiting the advantage of excluding NMS. In this paper, we propose the **Real-Time DETection TRansformer (RT-DETR)**, the first real-time end-to-end object detector to our best knowledge that addresses the above dilemma. We build RT-DETR in two steps, drawing on the advanced DETR: first we focus on maintaining accuracy while improving speed, followed by maintaining speed while improving accuracy. Specifically, we design an efficient hybrid encoder to expeditiously process multi-scale features by decoupling intra-scale interaction and cross-scale fusion to improve speed. Then, we propose the uncertainty-minimal query selection to provide

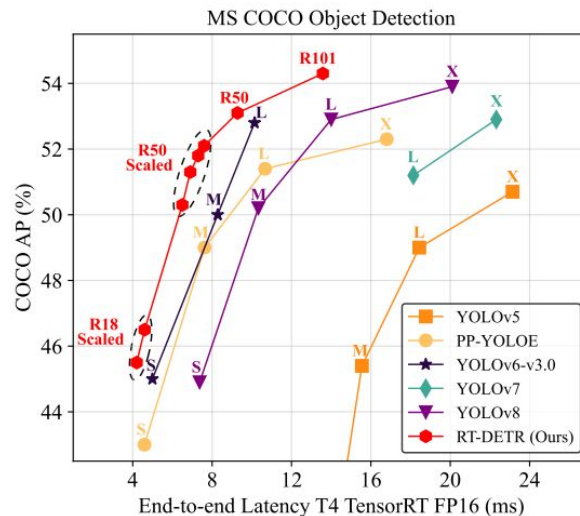
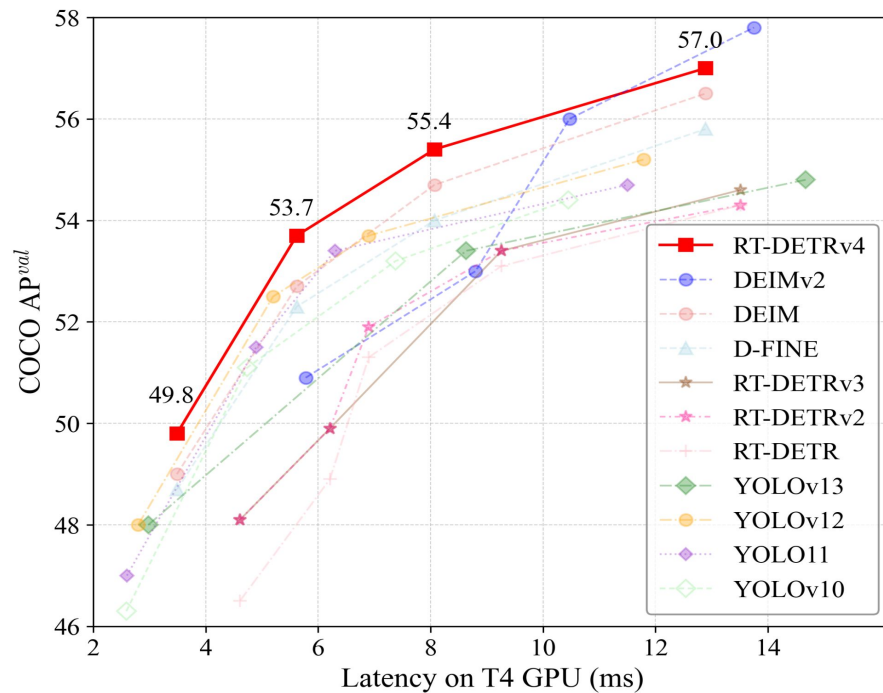


Figure 1. Compared to previously advanced real-time object detectors, our RT-DETR achieves state-of-the-art performance.

to their reasonable trade-off between speed and accuracy. However, these detectors typically require Non-Maximum

RT-DETRv4

→ <https://github.com/RT-DETRs/RT-DETRv4>



RT-DETR (Real-Time DETection TRansformer)

- Why Transformer ? Why RT-DETR ? as the alternative model
- YOLO is CNN-based model.
 - ◆ Good at Local Feature Extraction
- Transformer (Self-attention Mechanism)
 - ◆ Good at Capturing Global Context at once
 - ◆ High Accurate (due to capture global context)
 - ◆ NMS-Free
 - ◆ (But Early version cannot run on Edge due to larger model size)
- Compare RT-DETR vs YOLO
 - ◆ YOLO might miss the severe occlusion images. (Local Grid has no sufficient good pixels)
 - ◆ RT-DETR can relieve that due to global attention.

CNN vs Transformer model in ITS

Project Constraint	YOLOv10/v12 (CNN / Local)	RT-DETR (Transformer / Global)
Best Use Case	Highways, Toll Booths, Parking Lots	Complex Urban Intersections, Roundabouts
Primary Strength	Raw FPS, High localization for License Plates	Severe occlusion, contextual reasoning
Primary Weakness	Struggles when targets are heavily blocked	High memory bandwidth requirement
Hardware Target	Jetson Nano / Orin NX (15W - 25W)	Jetson AGX Orin / Thor (40W - 60W+)
Latency Profile	Sub-10ms (Ultra-Low)	15ms - 25ms (Moderate)

Key Metrics

- mAP@50: The mAP calculated at an IoU threshold of 0.5 (Not Strict Boundary)
- mAP@50-95: The average mAP calculated at various IoU thresholds from 0.5 to 0.95 in 0.05 increments (Rigorous Boundary)
- Recall: Measures the ability to detect all actual objects in the image.
- Intersection over Union Threshold (IoU)

Evaluation Metrics

- AP (Accurate Precision) :
 - ◆ Area Under the Precision-Recall curve for 1 Class*
- mAP (mean Accurate Precision) :
 - ◆ Area Under the Precision-Recall curves for all Classes*
- Precision
 - ◆ $\text{Precision} = \text{True Positive} / \text{Total Predictions}$
 - ◆ $\text{Total Predictions} = (\text{True Positive} + \text{False Positive})$
 - ◆ False Positive means “False alarm”
- Recall
 - ◆ $\text{Recall} = \text{True Positive} / \text{Total Ground Truths}$
 - ◆ $\text{Total Ground Truth} = (\text{True Positive} + \text{True Negative})$
 - ◆ True Negative means “miss target”

Where “TruePositive” is defined by $\text{IoU} \geq 0.5$ (or higher)

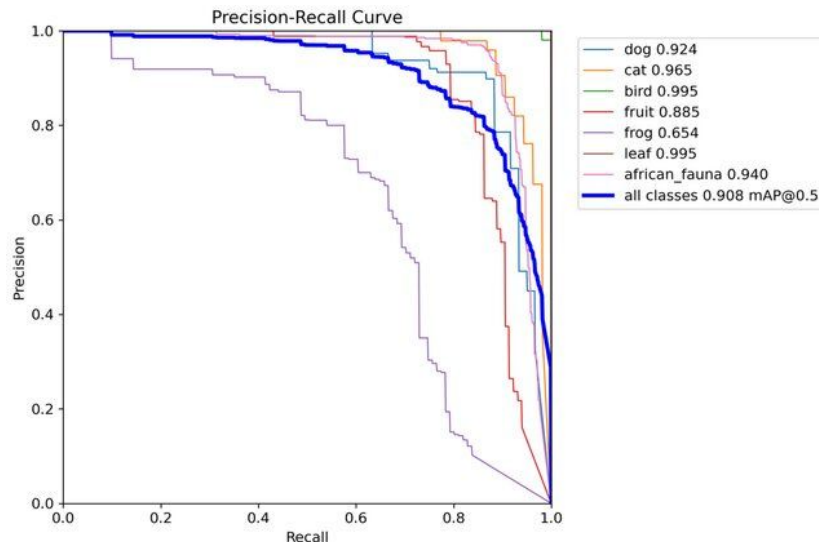


Figure from EDICA: A Hybrid Ensemble Architecture Using Deep Learning Models for Fine-Grained Image Classification

Section 3 : Implementation

Prepare stuffs

- YOLO : <https://github.com/ultralytics/ultralytics>
 - YOLO26 COLAB Tutorial :
<https://colab.research.google.com/github/ultralytics/ultralytics/blob/main/examples/tutorial.ipynb>
 - *Feasible to be used on Small device (Jetson Nano)
-
- RT-DETRv4 : <https://github.com/RT-DETRs/RT-DETRv4>
 - RT-DETR COLAB Tutorial :
<https://colab.research.google.com/github/roboflow-ai/notebooks/blob/main/notebooks/train-rt-detr-on-custom-dataset-with-transformers.ipynb?ref=blog.roboflow.com>
 - *Recommend to be used on Medium-to-High Spec. device (Jetson Orin)

YOLO, RT-DETR Init.

- # 1. Install the framework
- pip install ultralytics
-
- # 2. Import libraries
- from ultralytics import YOLO, RTDETR
- import torch
-

YOLO

```
# Initialize YOLO (Nano version for the Jetson Nano)

→ yolo_model = YOLO("yolov10n.pt")

# Or yolov12n.pt depending on library version

# Train for 20 epochs

→ yolo_results = yolo_model.train(
→     data="your_custom_data.yaml",
→     epochs=20,
→     imgsz=640,
→     batch=16,
→     device=0,
→     project="ITS_Workshop",
→     name="YOLO_Run"
→ )
```

RT-DETR

```
# Initialize RT-DETR (ResNet18 backbone for speed)

→ rtdetr_model = RTDETR("rtdetr-resnet18.pt")

# Train for 20 epochs

→ rtdetr_results = rtdetr_model.train(
→     data="thai-traffic-micro/data.yaml",
→     epochs=20,
→     imgsz=640,
→     batch=16,
→     device=0,
→     project="ITS_Workshop",
→     name="RTDETR_Run"
→ )
```

Validation

```
# Validate YOLO
```

```
print("--- YOLO Metrics ---")
```

```
yolo_val = yolo_model.val()
```

```
# Validate RT-DETR
```

```
print("--- RT-DETR Metrics ---")
```

```
rtdetr_val = rtdetr_model.val()
```

Deploy on Small Device (Ex. Jetson Nano)

- Export the model in proper format / size*
- For example, Export YOLO to TensorRT (FP16 Quantization for 2x speed boost)

- `yolo_model.export(`
- `format="engine",`
- `device=0,`
- `half=True, # Enables FP16 precision`
- `workspace=4 # Allocate 4GB of RAM for the conversion`
- `)`
-
- `# You now have 'yolov10n.engine'`

Example YOLO Script (Run on Jetson Nano)

```
→ from ultralytics import YOLO
→ import cv2
→ import time
→
→ # Load the compiled TensorRT Engine
→ model = YOLO("yolov10n.engine", task="detect")
→
→ # Open a test video (e.g., Sukhumvit road footage)
→ cap = cv2.VideoCapture("test_traffic.mp4")
→
→ while cap.isOpened():
→     success, frame = cap.read()
→     if not success:
→         break
→
→     start_time = time.time()
→
→     # Run inference
→     results = model(frame, verbose=False)
→
→     # Calculate Latency
→     latency_ms = (time.time() - start_time) * 1000
→
→     # Draw results
→     annotated_frame = results[0].plot()
→
→     # Add hardware metrics to screen
→     cv2.putText(annotated_frame, f"Latency: {latency_ms:.1f} ms", (20, 50),
→                 cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
→
→     cv2.imshow("Jetson Nano Edge Inference", annotated_frame)
→     if cv2.waitKey(1) & 0xFF == ord("q"):
→         break
→
→ cap.release()
→ cv2.destroyAllWindows()
→
```

Demo* Part

Part 2 : Advanced Vehicle Re-identification (ReID)

Agenda

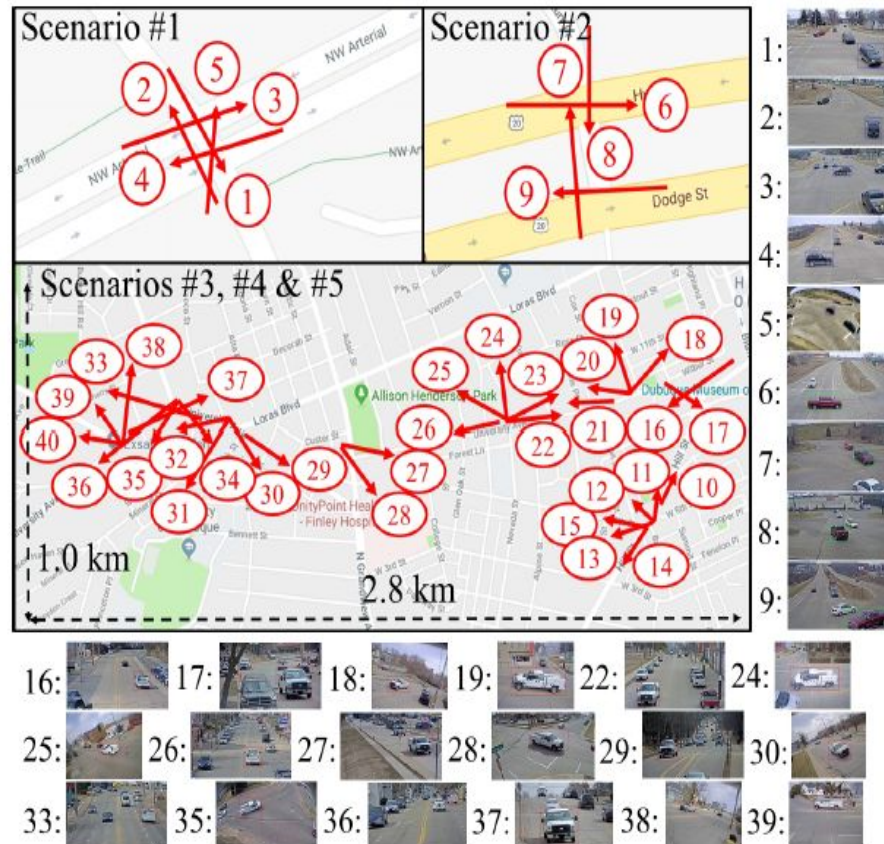
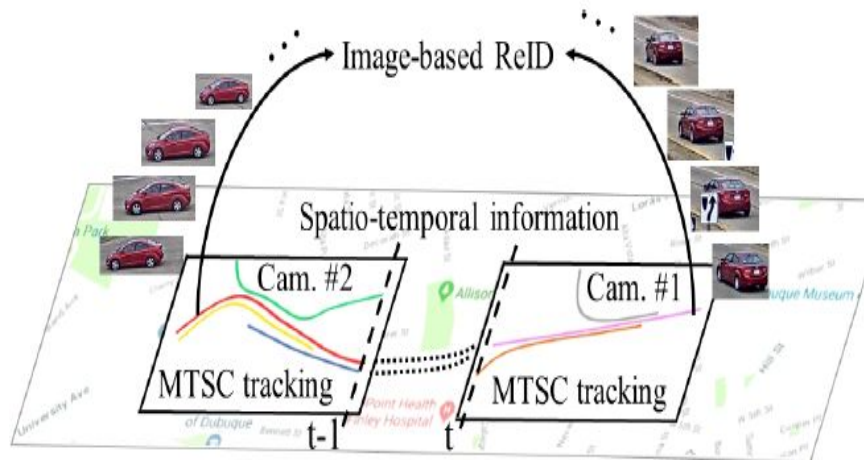
- Part 1 : The ReID Problem
- Part 2 : Transformer Architecture for ReID
- Part 3 : Basic ReID Framework
- Part 4 : City Level ReID Framework

ReID : The Identity across The City

ITS Needs : Challenges in Smart Cities

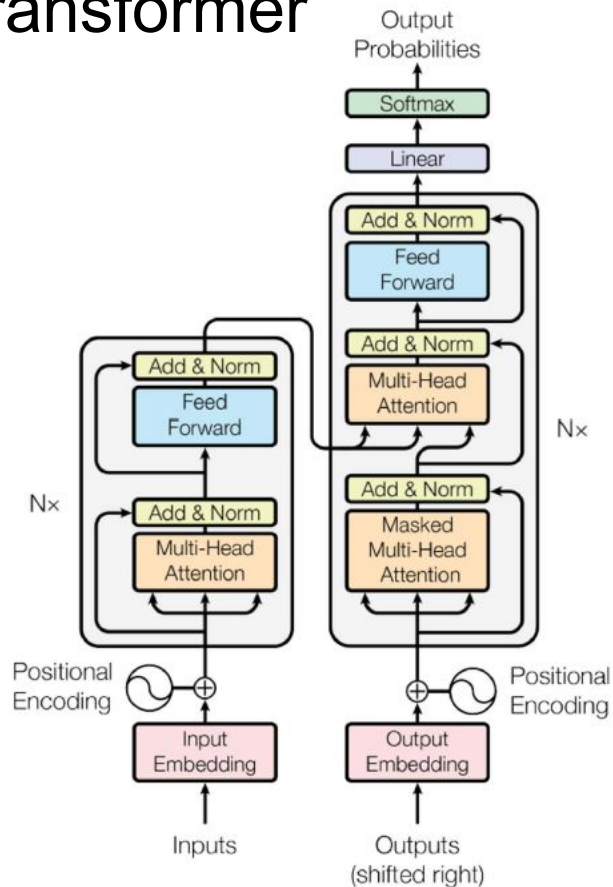
- Safer City !
 - Irresponsible Driving,
 - Distracted Driving,
 - Illegal Transportation,
 - Driving under Influence of Alcohol / Drugs
-
- Vehicle ReID enables the possibility to monitor, track and control traffic by leveraging the power of CCTVs.

Vehicle Re-Identification

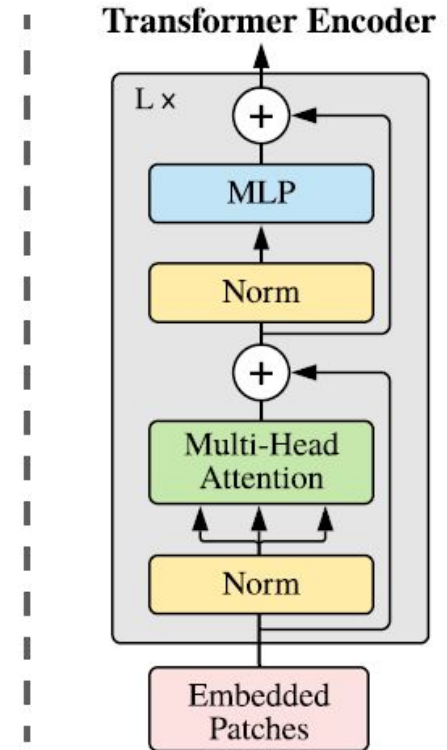
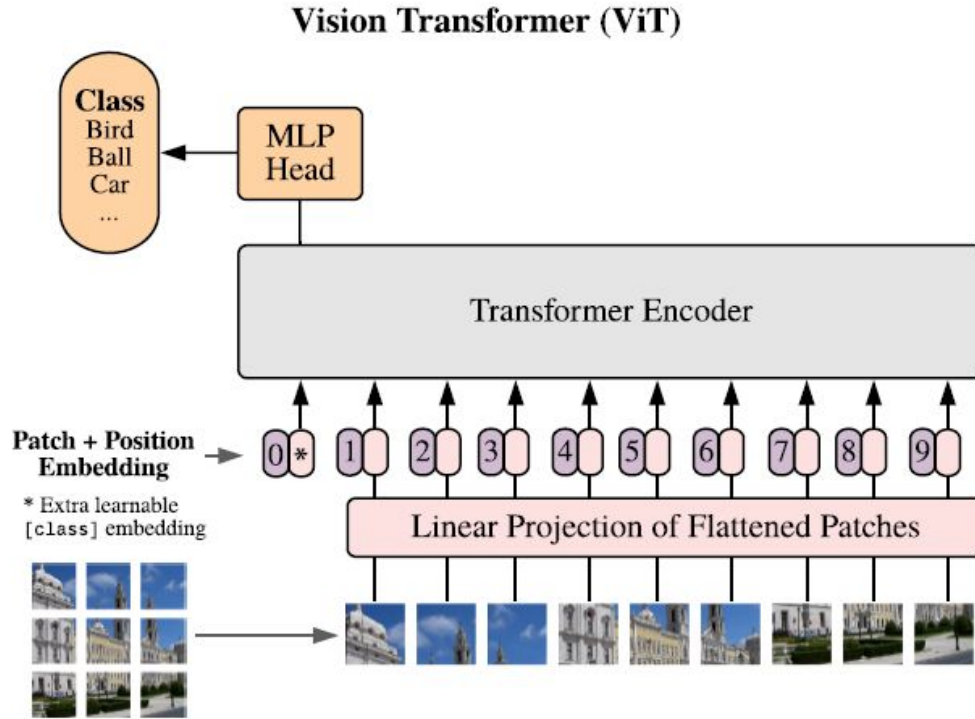


Transformer Architecture for ReID

The beginning of Transformer



Transformer Revolution



→ AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

Visualization of Attention in Transformer-based Method

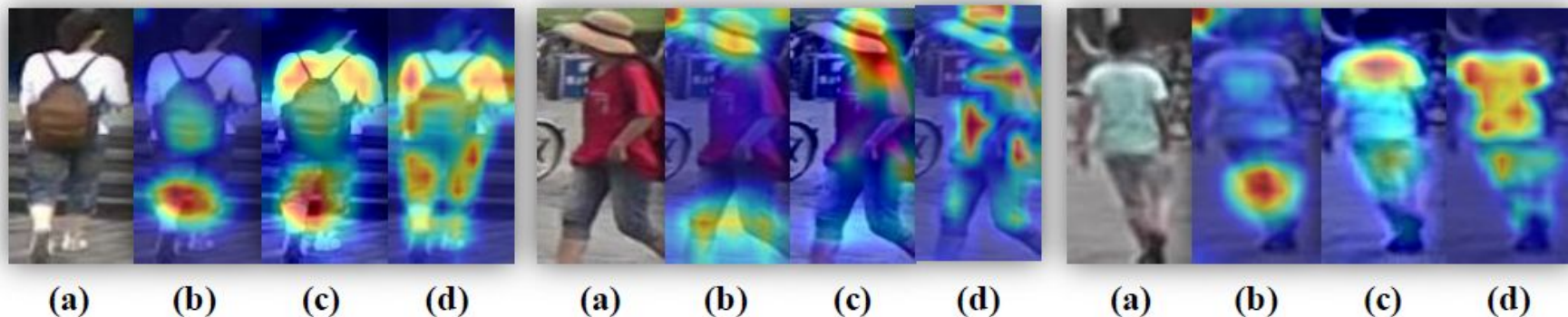
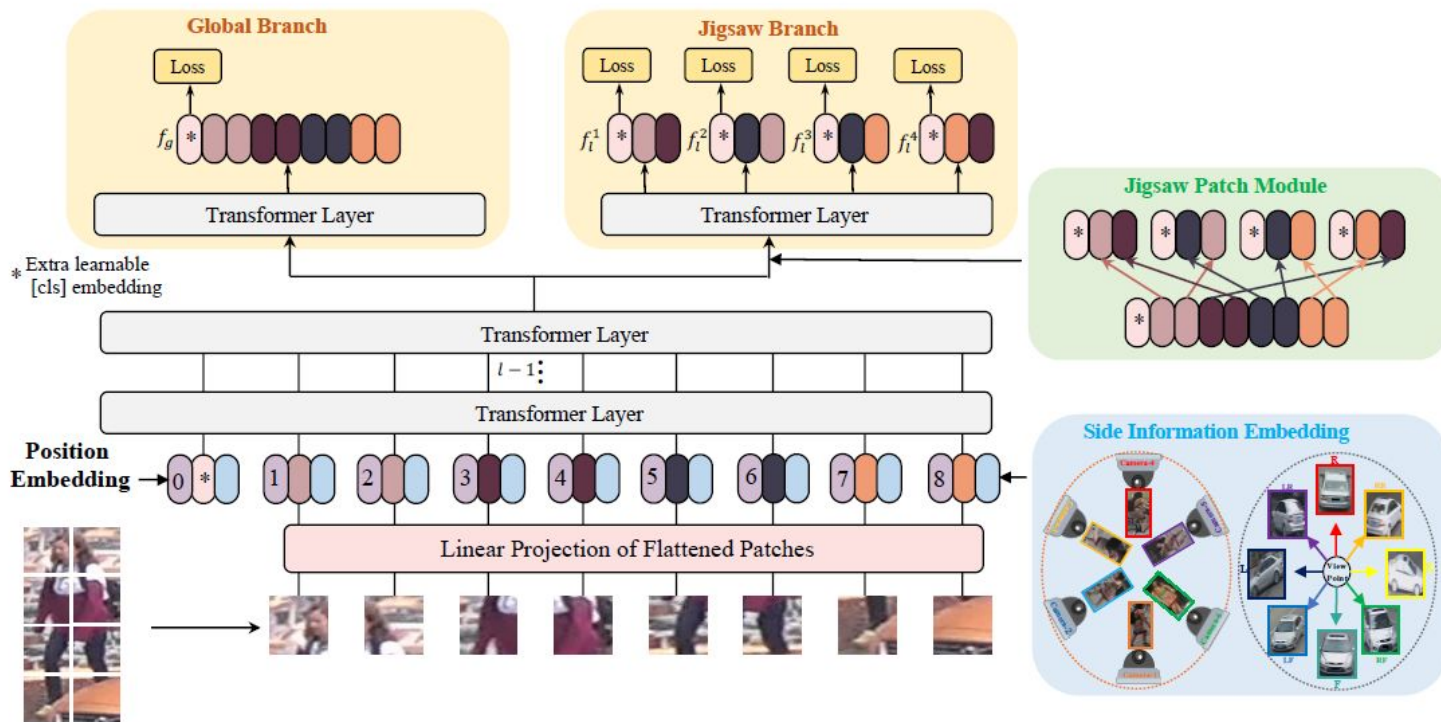


Figure 1: Grad-CAM [34] visualization of attention maps: (a) Original images, (b) CNN-based methods, (c) CNN+attention methods, (d) Transformer-based methods which captures global context information and more discriminative parts.

TransReID



→ TransReID: Transformer-based Object Re-Identification

Attention Map in TransReID

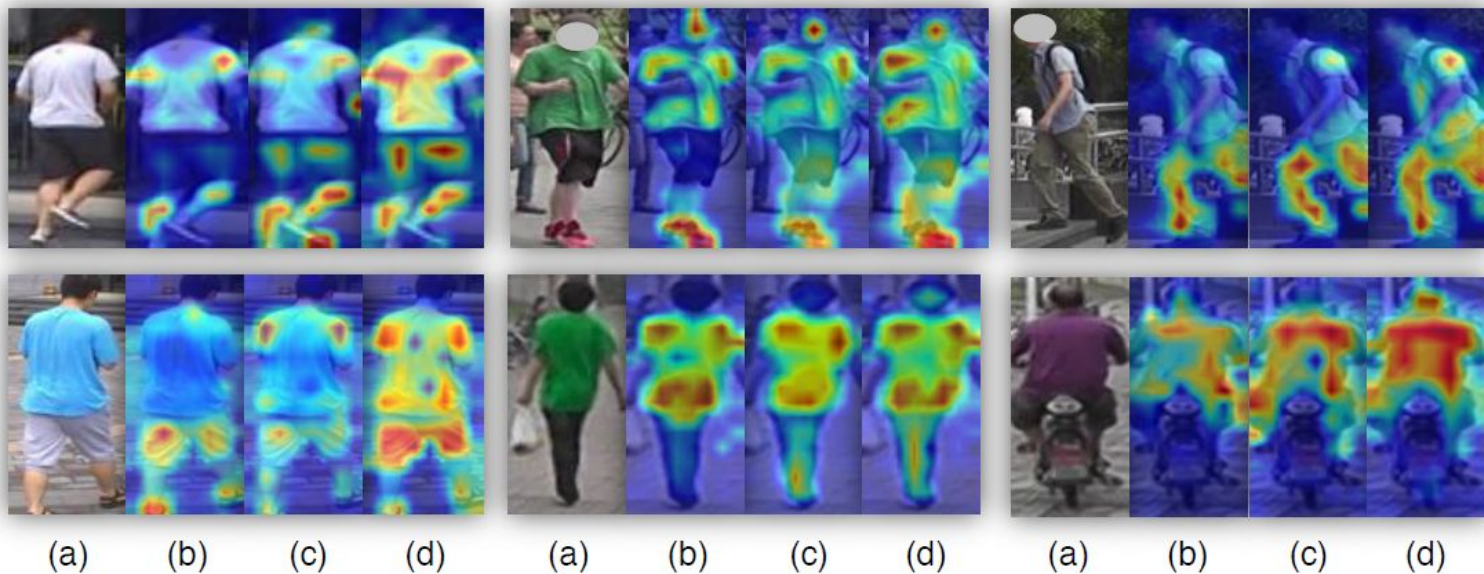


Figure 5: Grad-CAM visualization of attention maps. (a) Input images, (b) Baseline, (c) JPM w/o rearrange, (d) JPM.

→ TransReID: Transformer-based Object Re-Identification

Vehicle ReID Timeline

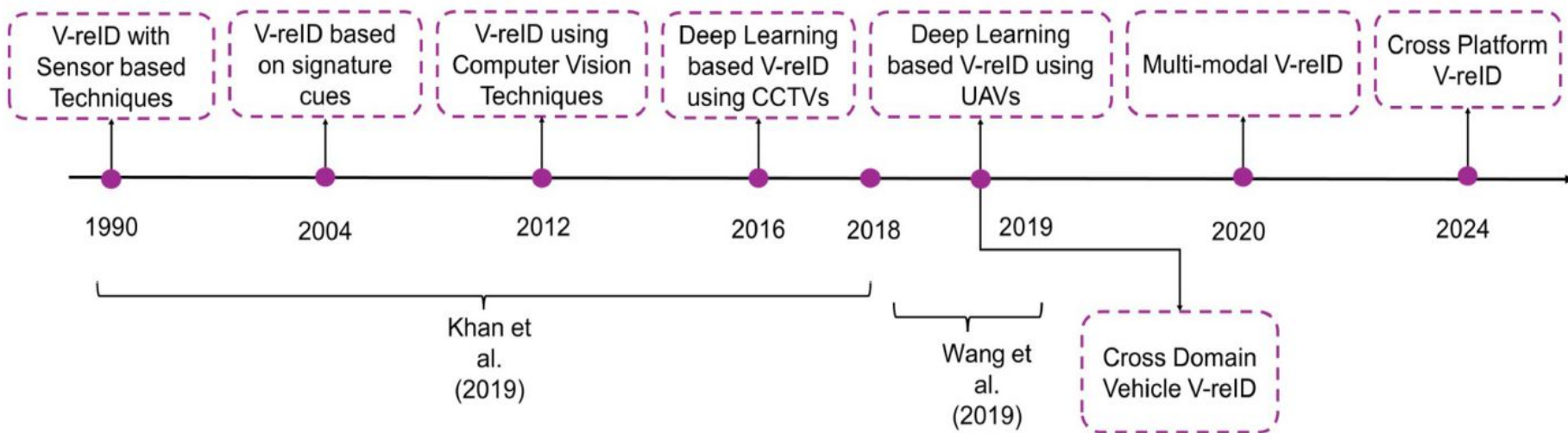


FIGURE 1. A timeline of the development of vehicle re-identification methods over the years.

Vehicle Taxonomy

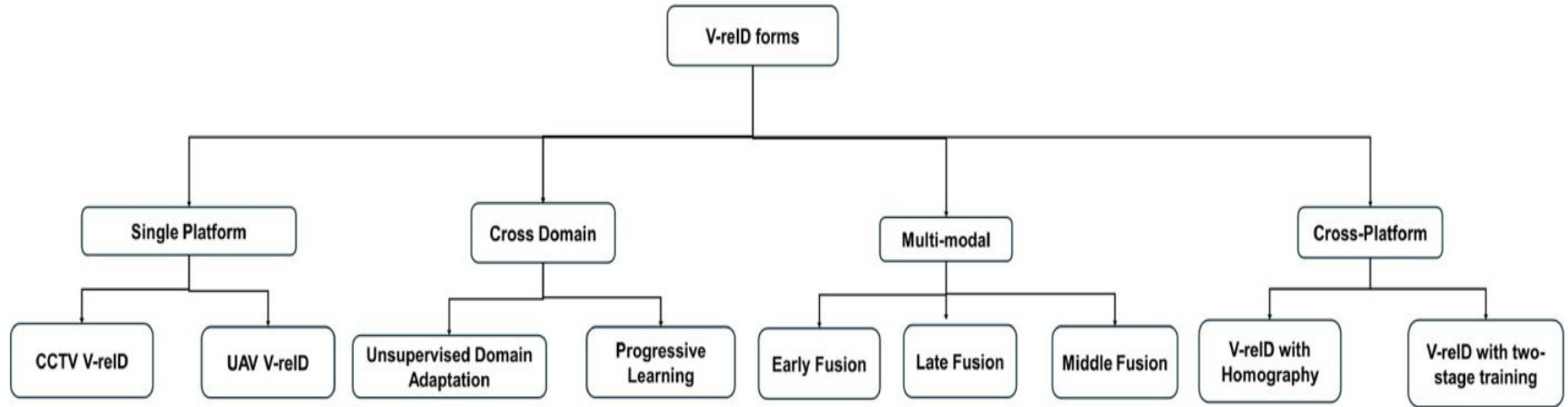


FIGURE 2. An illustration of the taxonomy of various forms in vehicle re-identification. Vehicle Re-identification modes are divided into Single Platform, Cross Domain, Multi-modal, and cross platform. Subcategories are provided for each mode that includes CCTV and UAV V-reID for Single Platform, Progressive Learning and Unsupervised Domain Adaptation for Cross Domain, and fusion techniques for Multi-modal. Furthermore, recent advancement in vehicle re-identification with cross platform mode contains categories of re-identification with Homography and Two-Stage Training.

Challenges in Vehicle ReID



FIGURE 4. Challenges faced during re-identifying the vehicles observed by cross platform surveillance systems. The identical vehicles (a) and (b) taken from MCU-VReID dataset [13] appears to be transformed/augmented version of one another presenting additional challenges in re-identification.

Normal Pipeline in Vehicle ReID

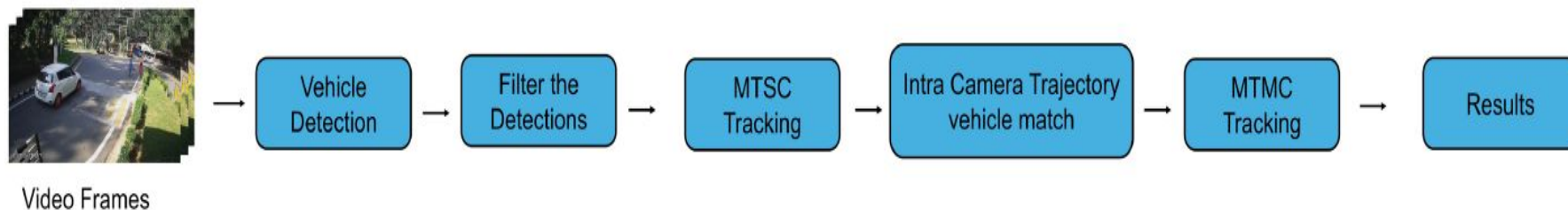


FIGURE 5. The vehicle tracking pipeline starts with vehicle detection, followed by filtering to remove incorrect detections. Within each camera, vehicle tracklets are created using single-camera tracking. These tracklets are then matched with tracklets from neighboring cameras for multi-camera, multi-target tracking.

Vehicle Re-ID Datasets

- COMPCARS
- VehicleID
- VeRi-776
- VehicleX
- MCU-VREID
- CityFlow
- RGBN300
- VRAI
- UAV-VeID

LARGE-SCALE VEHICLE RE-IDENTIFICATION IN URBAN SURVEILLANCE VIDEOS



Fig. 1. The urban surveillance environments and cameras distribution for the VeRi dataset. The blue circles and arrows denote the locations and directions of cameras. The surveillance camera views illustrate an example of vehicle Re-Id: the same white BMW SUV captured by the surveillance network.

VeRi-776 Datasets



(a)



(b)

Fig. 2. Vehicle samples selected from the VeRi dataset. All images are normalized to the same size. (a) Diversity of vehicle colors and types. (b) Variation of the viewpoints, illuminations, resolutions, and occlusions for the vehicles in different cameras.

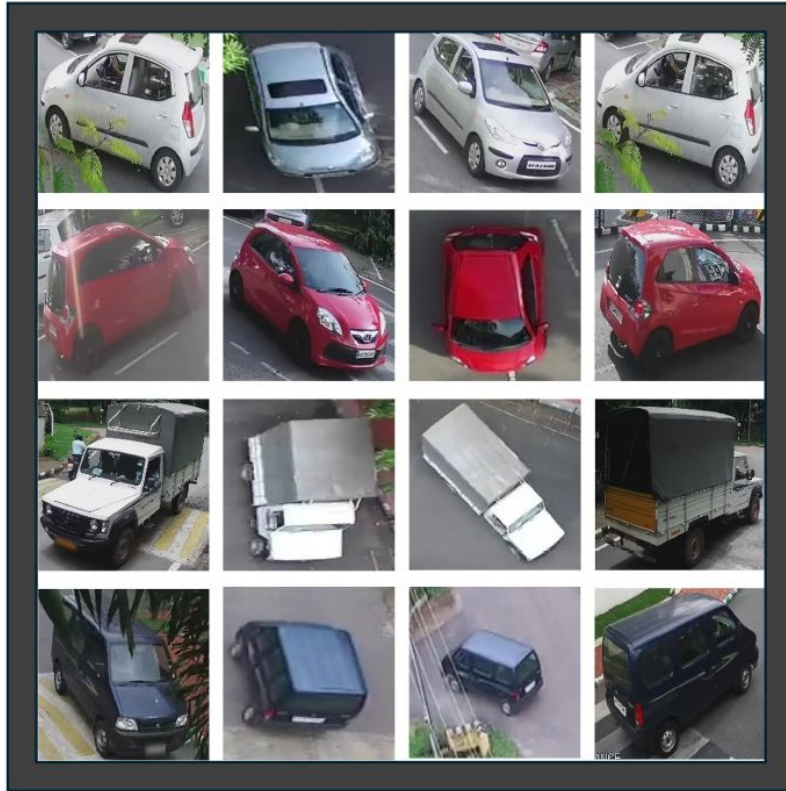
VeRi-776 Dataset



- 20 CCTVs
- 51,035 images
- 776 identical vehicles
- 218 viewpoints

FIGURE 6. Sample vehicle images from VeRi-776 dataset.

MCU-VReID



- 42 CCTVs + UAVs
- Use for Cross Platform
- 51 Identical Vehicles
- 5,630 Images

FIGURE 7. Sample images from MCU-VReID dataset.

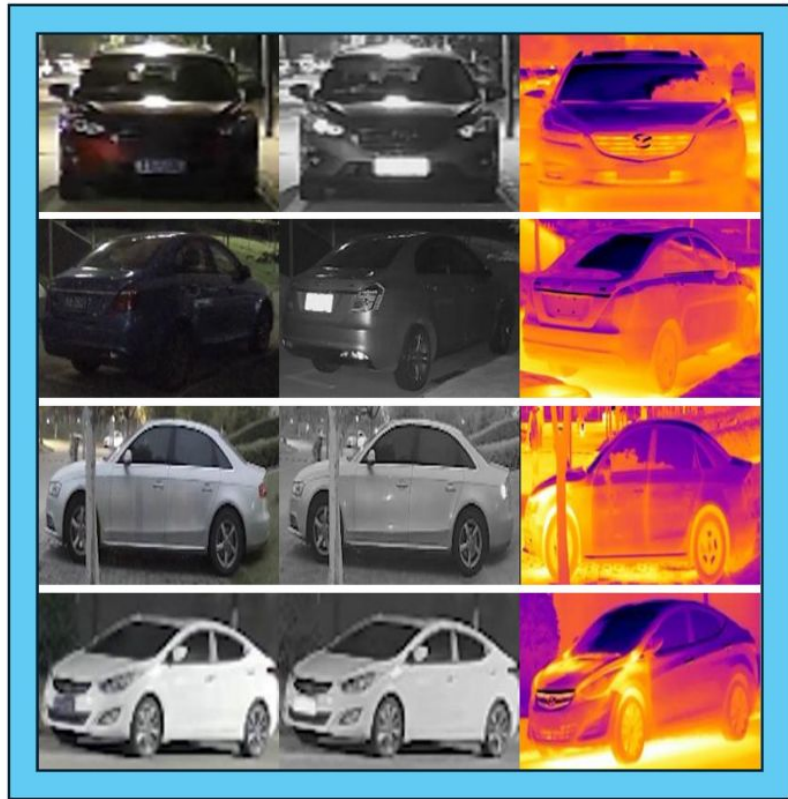
CityFlow Dataset

- 40 Cameras from 10 Intersection in US City
- Largest Dataset
- > 200,000 annotated BBox
- 666 identical vehicles
- Videos 960p + 10FPS
- Designed for research in MTMC vehicle tracking



FIGURE 8. Sample images form CityFlow dataset.

RGBN300 Dataset



- Multi-Spectral Images
- 300 Identical Vehicles
- 8 Camera Views
- 50,125 RGB images
- 50,125 NIR (Near InfraRed) images
- Additional 17,250 Thermal Infrared Images of 100 Vehicles

FIGURE 9. Sample images from RGBN300 dataset.

VRAI Dataset

- 350 video clips
- 13,022 identical vehicles
- Two UAVs
- Different Location, Flight Altitude and Angles



FIGURE 10. Sample images from VRAI dataset.

Basic ReID Framework : Fast Re-ID

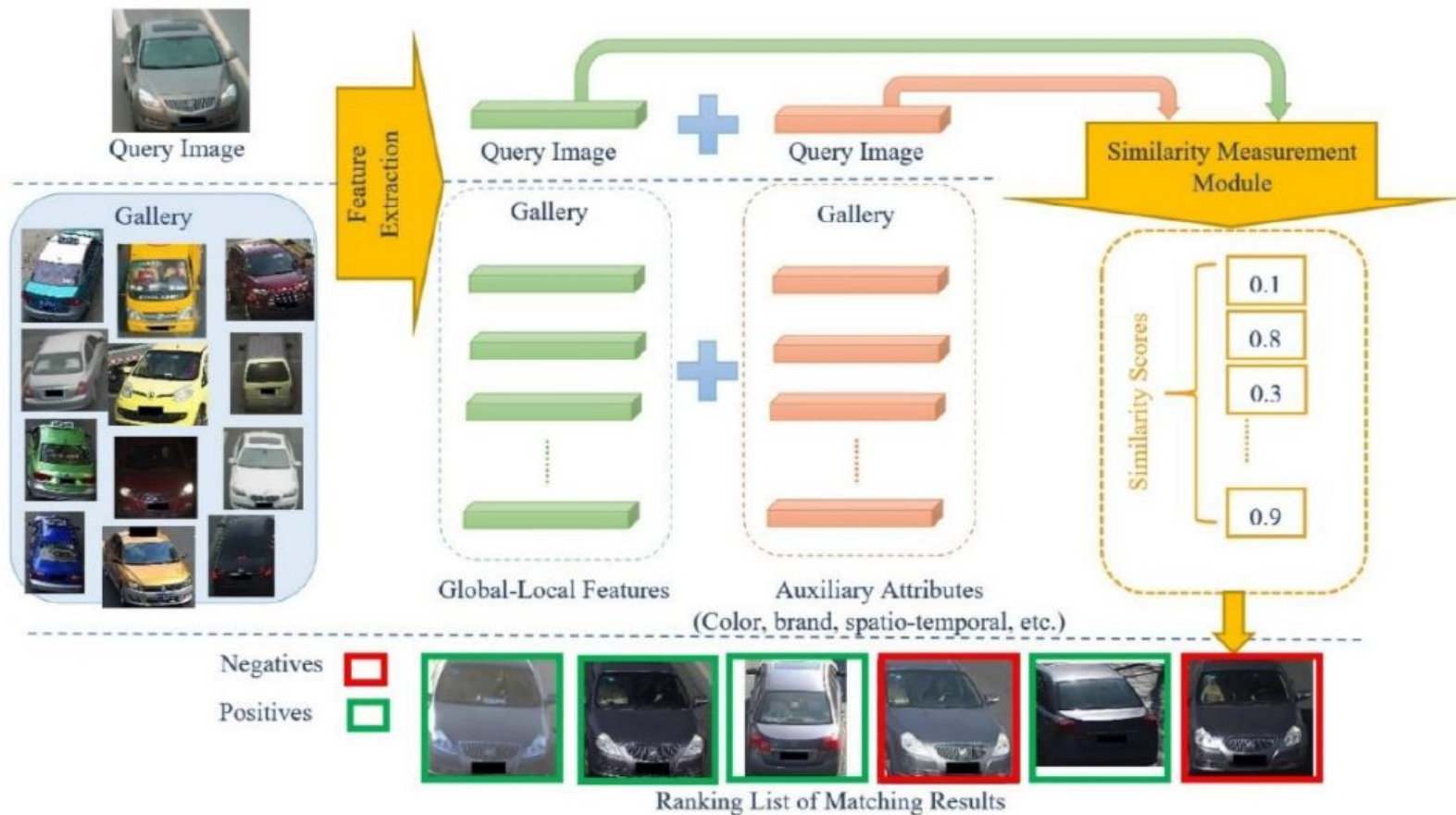


Fig. 1. Vehicle ReID description. The vehicle image's global and local features and additional auxiliary attributes are extracted and used to compare with gallery images to retrieve the ranking output list.

Pipeline of Fast ReID

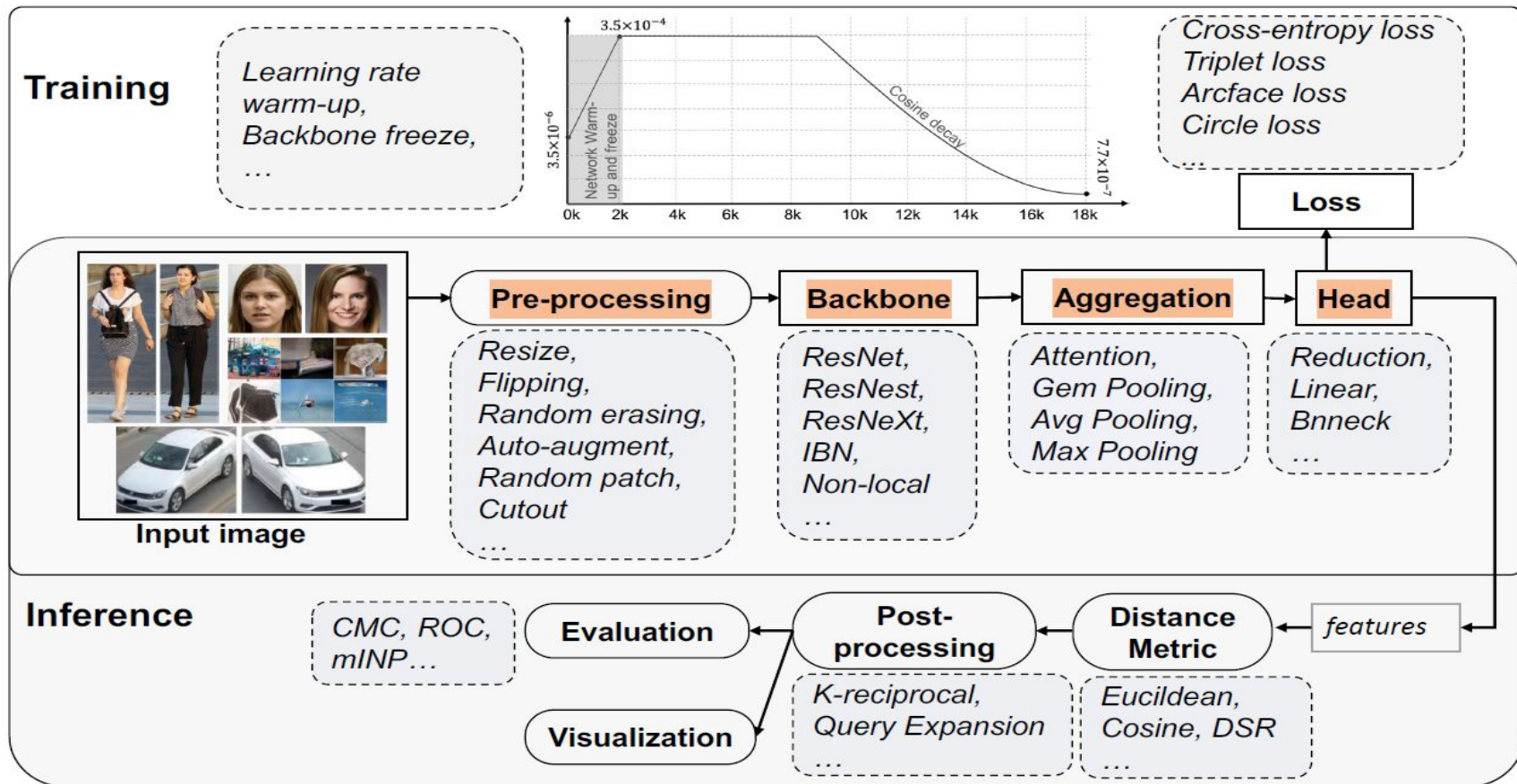


Figure 1. The Pipeline of FastReID library.

Pre-processing

- Resize
- Flipping
- Random Erasing
- Auto-Augment
- Random patch

Image Pre-processing & Augmentation

To make the model robust against environmental noise, we apply aggressive data augmentation:

-  **Resize & Padding:** Normalizing input dimensions.
-  **Random Flip:** Simulating different camera angles.
-  **Random Erasing:** Simulating occlusions (e.g., other cars).
-  **Auto-Augment:** Learning optimal augmentation strategies.

Random Erasing Augmentation

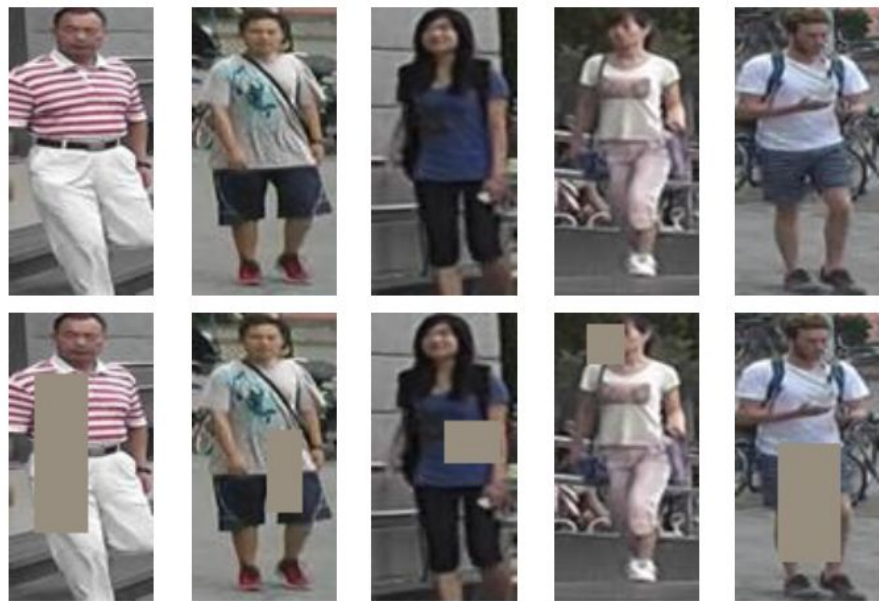


Figure 4. Sampled examples of random erasing augmentation. The first row shows five original training images. The processed images are presented in the second row.



Feature Extraction & Aggregation



Backbone

Deep CNNs (ResNet, ResNeSt, IBN) extract multi-scale spatial feature maps from the augmented image crops.



Aggregation

Collapses $H \times W$ feature maps into a 1D vector using Global Average Pooling (GAP), Max, or GeM Pooling.



Attention

Self-attention mechanisms focus on discriminative parts of the vehicle (e.g., headlights, stickers, dents).

Backbone (CNN)

- ResNet
- ResNest
- ResNeXt
- IBN
- Non-Local

Aggregation

- Collapse 3D Tensor ($C \times H \times W$) → 1D vector ($C \times 1$)
- Global Avg Pooling
 - ◆ Average all pixels in each channel
 - ◆ ReID : GAP → Global Summary of the image
- Max Pooling
 - ◆ Max all pixels in each channel
 - ◆ ReID : MAP → Capture High Outstanding Features (Bright glow of distinct vehicle parts)
- GeM (Generalized Mean) Pooling
 - ◆ Balance between GAP and MAP
 - ◆ Dynamic Finding optimal parameter p that define how much details the model need between GAP and MAP
- Attention Pooling
 - ◆ Avoid blindly treat all pixels equally : Attention Map $H \times W$
 - ◆ Masked pixels by Attention before GAP / MAP

Head

- Reduction Layer
- Linear Layer
- Bnneck Layer

Reduction Layer

- Spatial Pooling → Distance matrix can be computed.
- Reduce 3D Tensor ($C \times H \times W$),
 - ◆ e.g., ResNet50 = $2048 \times 16 \times 8 \rightarrow 1D$ vector
- Global Average Pooling (GAP) → For each channel c , compute the average of all pixels in the Feat Map.
- Reduce to 1D Vector ($C \times 1$)
 - ◆ E.g., ResNet50 = $2048 \times 16 \times 8 \rightarrow 2048 \times 1$ vector*

Linear Layer

- Sometimes, the number of channels (in Feature maps) are too high for computation*.
- Reduce it :
 - ◆ ResNet50 : 2048 dimensional vector
 - ◆ Reduce to 512 dimension*
- How ?
 - ◆ Linear Projection by Fully-connected Network

The Re-ID Head: BNNeck Design

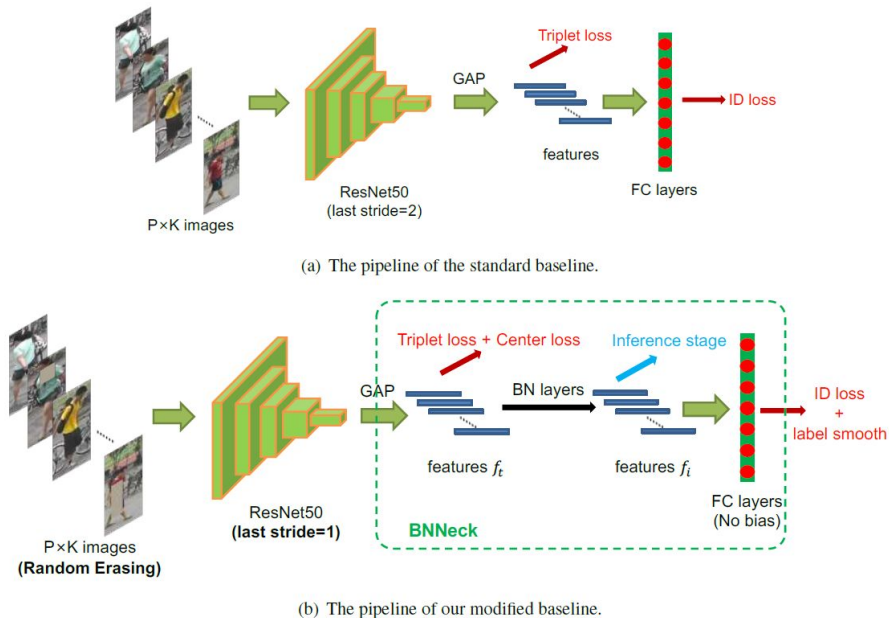


Figure 2. The pipelines of the standard baseline and our modified baseline.

BNNeck Innovation

Standard Re-ID heads often struggle to balance Classification Loss and Triplet Loss simultaneously. BNNeck solves this by:

- Separating feature spaces for different losses.
- **Before BN:** Feature used for Triplet Loss (distance).
- **After BN:** Feature used for Cross-Entropy (ID).

BNNeck in ReID

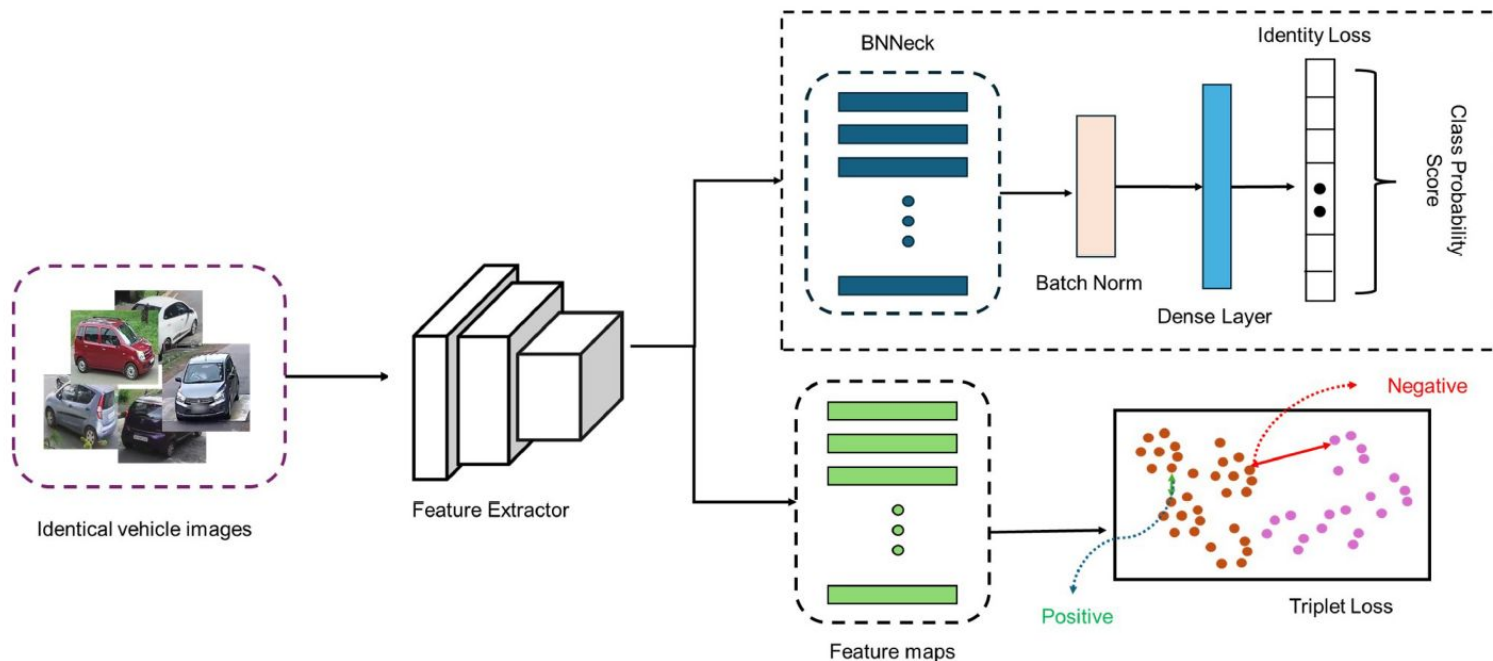


FIGURE 3. Illustration of different phases in a vehicle re-identification architecture: A feature extractor processes identical vehicle images to produce feature maps. These feature maps are generally fed to two branches, i.e., (a) a classification branch with Bottleneck, batch normalisation, and a dense layer to compute class probabilities using identity loss. (b) Metric learning branch using triplet loss to bring positive samples closer in the embedding space while separating negatives.

BNNeck (Batch Normalization Neck)

- Problem *
- Contradiction between Triplet Loss and CE Loss
 - ◆ Triplet loss optimizes distance in Euclidean space.
 - ◆ CE loss optimizes vectors into another space (hypersphere).
- Solution : BNNeck
 - ◆ To prevent Gradient Oscillation by Splitting Loss Calculation
 - ◆ Apply Triplet Loss before BNNeck (Euclidean Distance of Raw Vector)
 - ◆ Apply CE Loss after BNNeck (Normalized (Tighter constrained) for Hyperplane)

Optimization: Ranking & Identity



Triplet Loss Formula:

$$L = \sum [d(a, p) - d(a, n) + \alpha]_+$$

Loss Functions (Used in ReID)

- Cross-entropy loss
- Triplet loss
- Arcface loss
- Circle loss

Cross Entropy (CE) Loss Function

- Why CE Loss function ?
- Attempt to identify each object categorized by ID (same as classification that classifying each object into its own class)
- Final output layer (will look alike this softmax ...)
 - ◆ $P_i = e^{x_i} / \sum_{j=1}^C e^{x_j}$ where x_i is raw score and C is total classes.
- Cross Entropy Loss :
 - ◆ $L_{\{CE\}} = - \sum_{i=1}^C y_i * \log(p_i)$
 - ◆
- Normally, In ReID, CE is used for ID Loss function

Triplet Loss

- Concept of “Classes”
- Used for optimizing Euclidean Distance between Feature vectors
- Look for 3 images at the same time :
 - ◆ Anchor (a) : Baseline image for Car X
 - ◆ Positive (b) : Different image for Car X
 - ◆ Negative (n) : Image of Car Y
- $L_{\text{triplet}} = \max(0, (\|f(a) - f(p)\|_2)^2 - (\|f(a) - f(n)\|_2)^2 + \alpha)$
- Distance between Anchor & Positive => should be small.
- Distance between Anchor & Negative => should be large
- L_{triplet} → Force the model to learn to ensure that negative images are farther away than positive images at least margin “alpha”

ArcFace Loss

- Firstly, Designed for Face Recognition
- Concept* :
 - ◆ Map all feature vectors onto a hypersphere
 - ◆ Calculate distances based on the angle between them
 - ◆ Penalty deviated angle to ground truth
 - ◆ Force Embeddings to be tighter
- Used in ReID for :
 - ◆ Create a highly compact cluster for each vehicle ID

Circle Loss

- Create to fix to reduce waste on computational effort in Triplet Loss.
- Computational effort loss :
- When Positive pairs are already (sufficient) close together → Triplet loss might still try to pull those pairs together.
- Circle Loss → Dynamic weighting positive and negative pairs.
 - ◆ When the pair is close enough, weighting is forced to drop.
 - ◆ Loss function does not attempt to waste computational effort and focus on other far-apart pairs.

Distance Matrix

→ Pairwise comparison between every query and every gallery

→ Definition :

- ◆ M : number of query images
- ◆ N : number of gallery images
- ◆ d : dimension of feature embeddings (e.g., $d = 2048$)
- ◆ Query Matrix (Q) : $M \times d$ (each row = query vector, q)
- ◆ Gallery Matrix (G) : $N \times d$ (each row = gallery vector, g)

→ Distance Matrix :

- ◆ $M \times N$ matrix where $D(i,j)$ denote to the distance between i -th query and j -th gallery

Similarity Search: Distance Metrics

Metric	Computational Cost	Use Case
Euclidean Distance	Low ($O(N)$)	Standard spatial similarity.
Cosine Similarity	Medium	Directional feature alignment.
Jaccard Distance	High	Re-ranking with neighbor sets.

Cosine Similarity:

$$\text{Similarity} = \frac{A \cdot B}{\|A\| \|B\|}$$

Distance matrix

- Euclidean
- Cosine

Euclidean Distance (L2)

- Straight Line distance between two points in N-dimension
- $d_E(x,y) = \sqrt{\sum_i (x_i - y_i)^2}$
- $d_E(x,y) = \|x - y\|_2$ or (L2 norm)
- Note : Euclidean distance is highly sensitive to magnitude (which may be dependent on light / shadow / contrast) of the image. (Normalization required)

Cosine Distance

- Do not measure the distance.
- Measure the angle between two N-dimensional vectors
- $\text{Cosine_Similarity}(x,y) = (x \cdot y) / \|x\|_2 * \|y\|_2$
- Similarity $\rightarrow 1$ means two vectors are almost identical.
- $\text{Cosine_distance}(x,y) = 1 - \text{Cosine_Similarity}(x,y)$

Post-Processing

- Re-Ranking
- K-reciprocal
- Query Expansion

Metrics for Evaluation

- CMC Rank-1, 5, 10
- ROC
- mAP

City Scale Re-ID Example #1

City-Scale Multi-Camera Vehicle Tracking Guided by Crossroad Zones

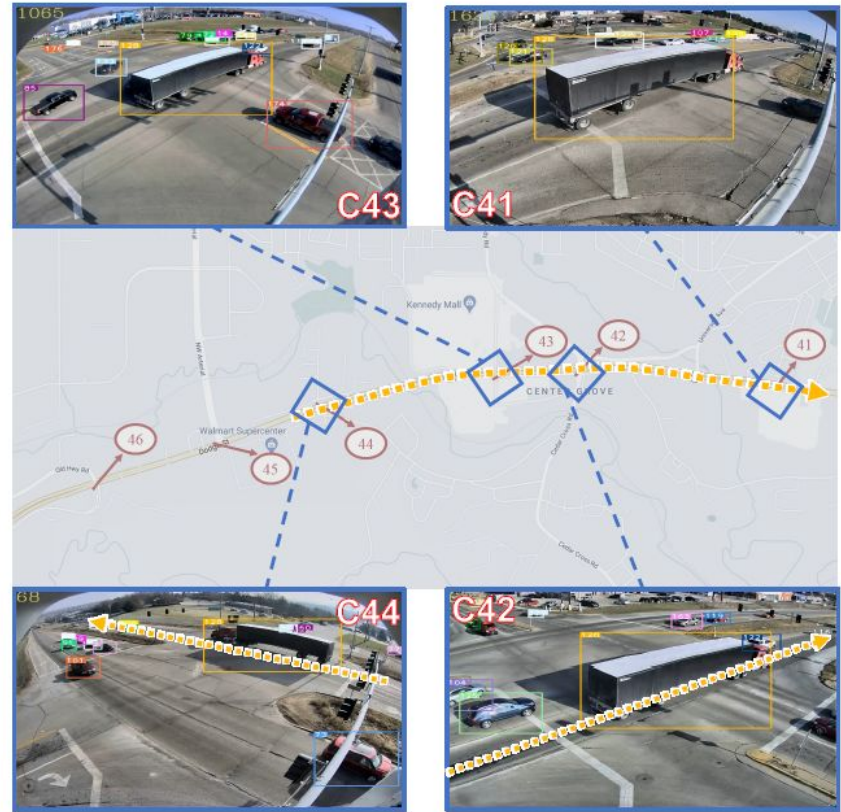


Figure 1. **Multi-camera vehicle tracking.** Multi-camera multi-target vehicle tracking requires finding the same vehicle that appears in multiple cameras. Their appearance and size usually vary greatly depending on the angle of view and the distance from the camera.

MTMC Pipeline

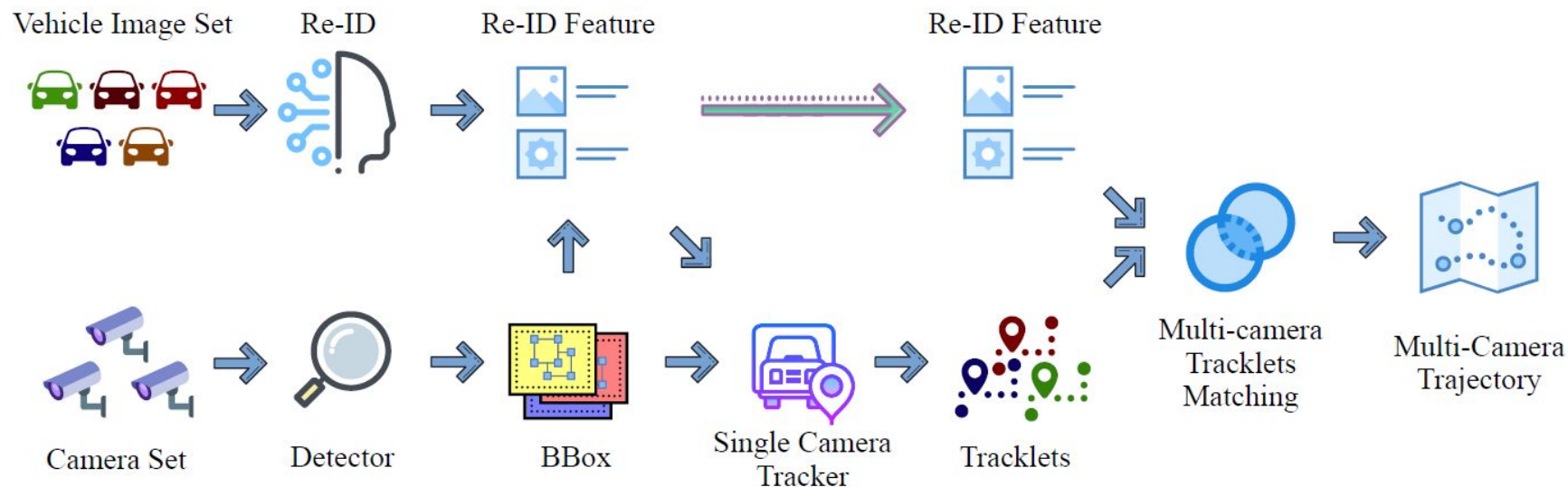
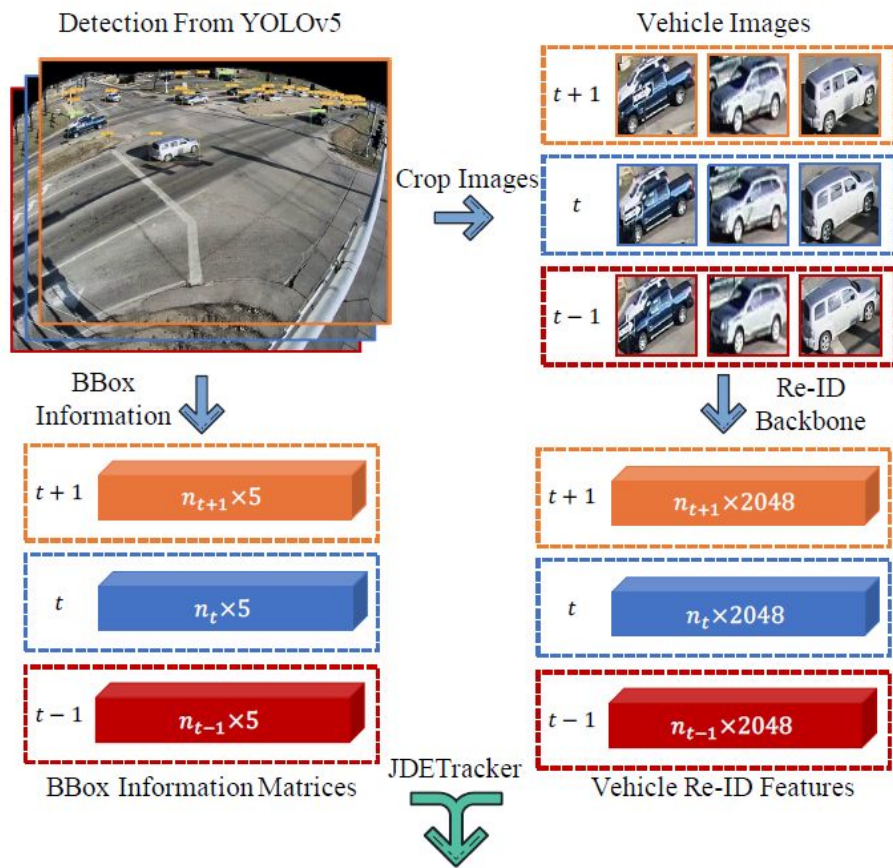


Figure 2. **Pipeline of Multi-Target Multi-Camera.** The MTMCT system first uses the detector to obtain the BBox of the target from each frame of each camera video; then uses the trained Re-ID model to extract the apparent features of the target BBox; the single-camera tracker uses BBox and Re-ID feature for each tracking target to generate single-camera tracklets; finally, according to the single-camera tracklets and Re-ID feature, the ID synchronization between cameras generates a multi-camera trajectory.

Tracking in Single Camera



Single Camera Tracklets

Figure 3. **Vehicle single camera tracking framework.** The detector generates vehicle detection for each frame, and uses the Re-ID backbone to generate the corresponding detection appearance features. Modified JDETracker uses BBox information and Re-ID appearance features for single-camera tracking, and generates single-camera vehicle tracking for each camera.

Crossroad Zones

→ Zoning to determine the vehicle directions at each intersection

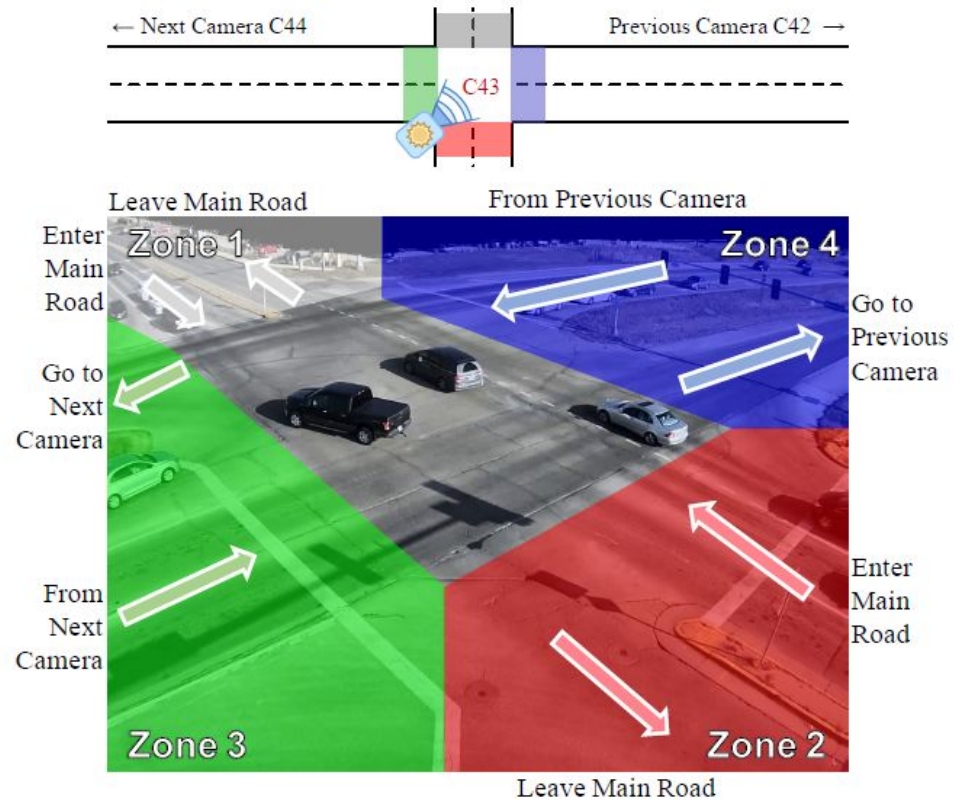


Figure 4. **Crossroad zones for C43.** According to the characteristics of the crossroad, we divide the video frame into four zones to divide the vehicle tracklets.

Multi-Camera Tracklets Matching

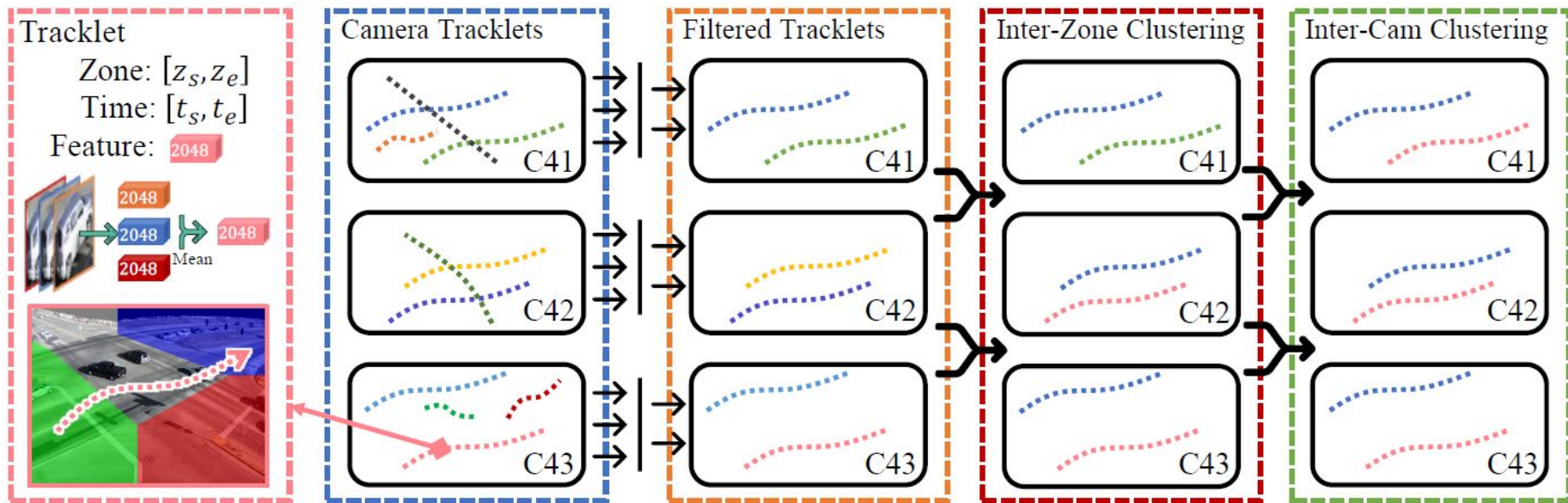


Figure 5. **Process of Multi-camera Tracklets Matching.** 1) Generate the necessary information for matching according to crossroad zones and tracklets; 2) Use TFS to filter the tracklets; 3) Calculate the similarity matrix between the tracklets and use DBTM to perform matching constraints; 4) Perform SCAC process between tracklets, perform inter-zone clustering and inter-cam clustering respectively.

Example



Figure 6. Visualization of vehicle multi-camera tracking results.

City Scale Re-ID Example #2 :
PROVID

Tracking Example

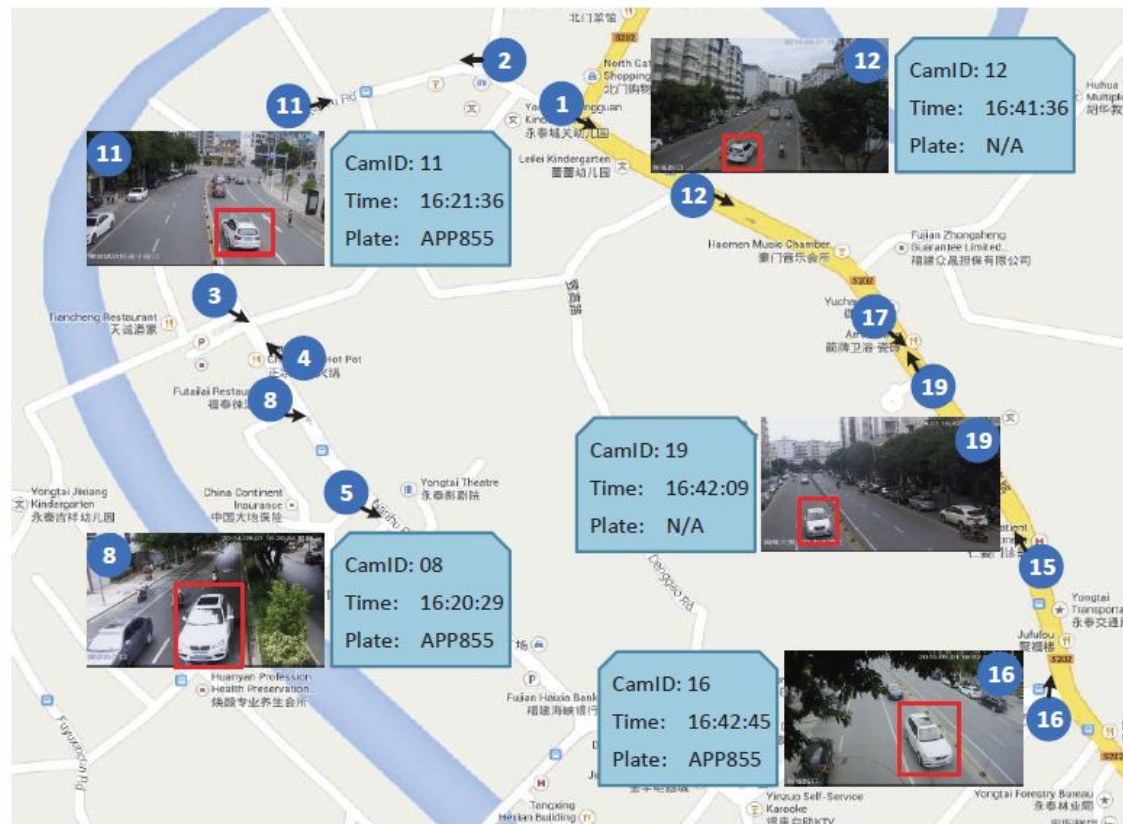


Fig. 1: An example of vehicle Re-Id: searching for the white BMW SUV with multi-modality data in urban surveillance.

Intra-Class Variance VS Inter-Class Similarity

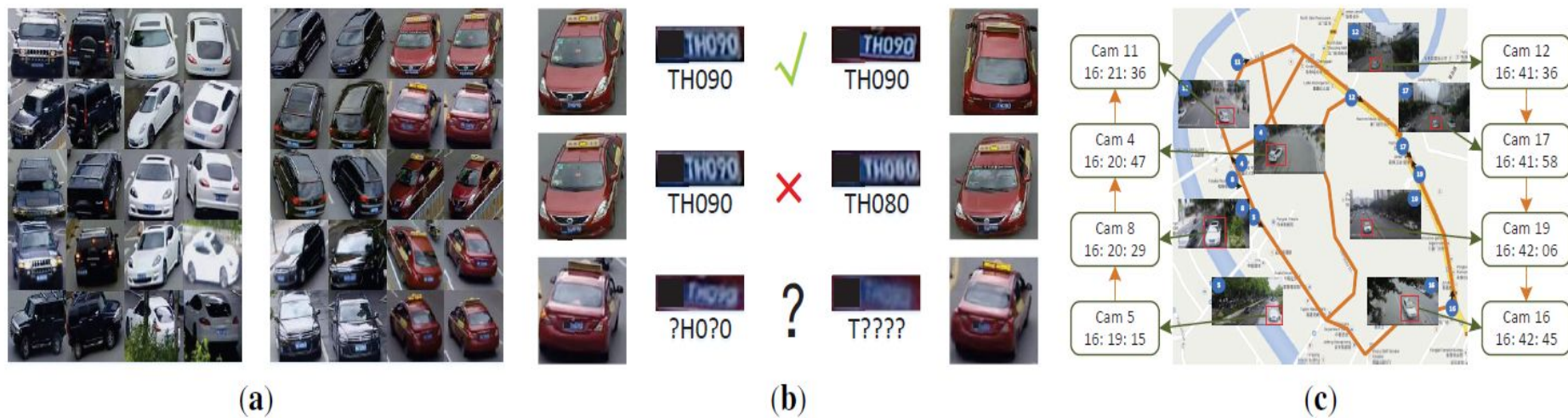


Fig. 2: (a) The same vehicles have great within-class differences in different viewpoints (left). Different but similar vehicles have trivial inter-class differences (right). (b) The license number plates as the unique ID for vehicle search. (Plate is masked to protect privacy.) (c) The contextual information can assist in vehicle search in the city.

Proposed Pipeline

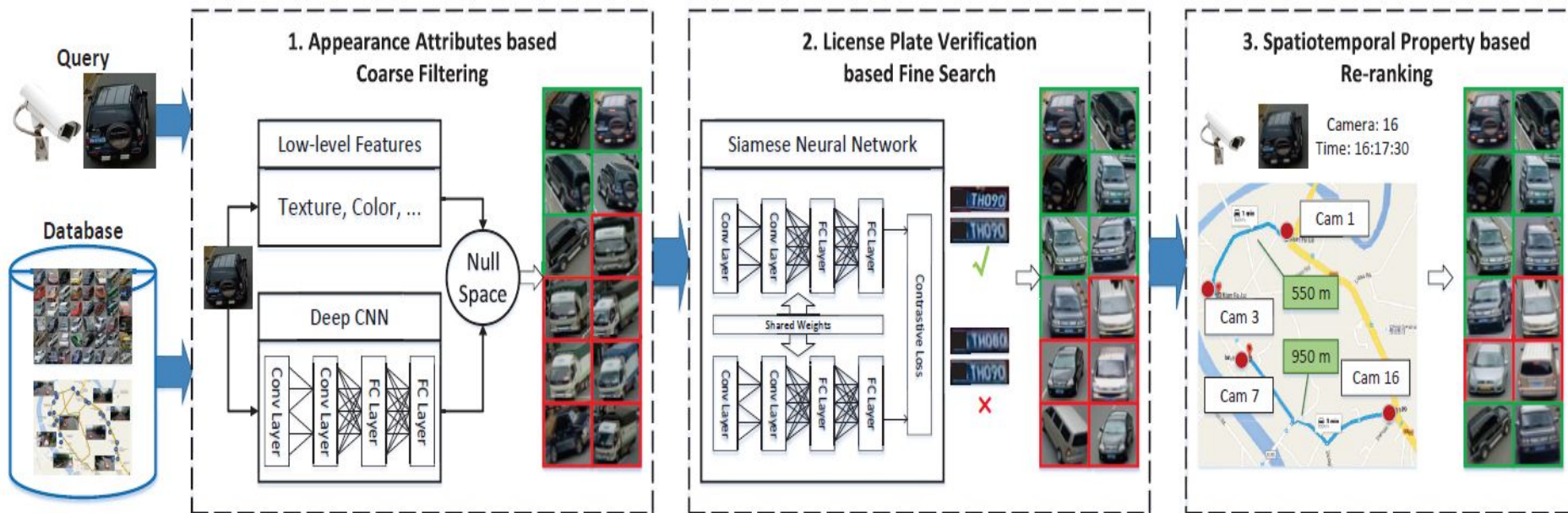


Fig. 3: The architecture of the PROVID framework.

Leverage Personal ID Vehicle Data

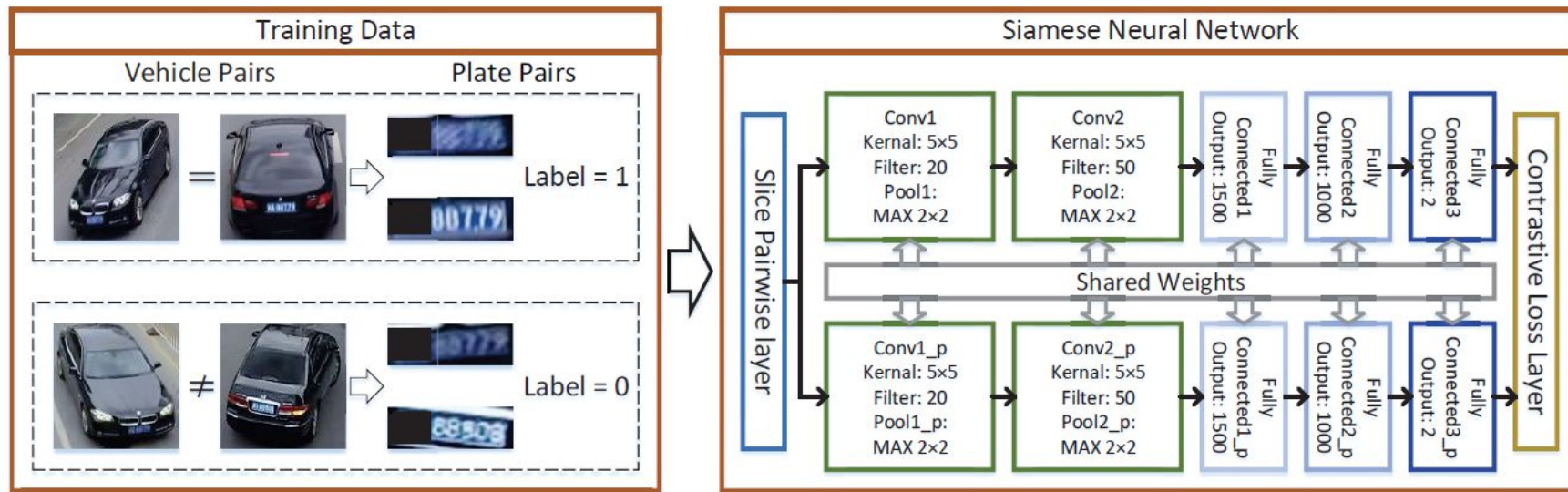


Fig. 5: The architecture of license plate verification based on the Siamese neural network.

Spatial and Temporal Data Integration

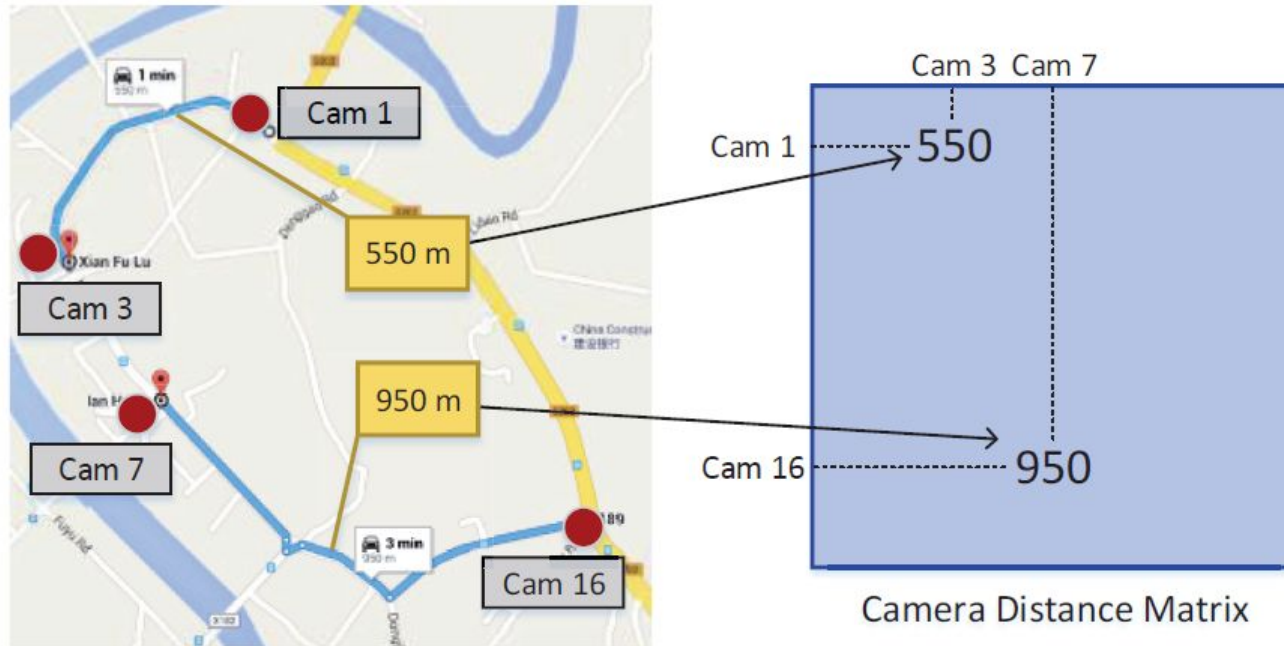


Fig. 7: The physical distance matrix of surveillance cameras.